

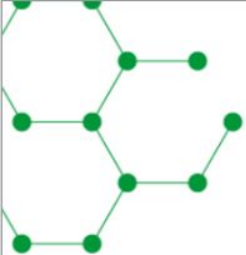
Conceitos e Aplicações de Aprendizado de Máquina

Aula 2: Aprendizagem Supervisionada

Eduardo Krempser
Assessoria de Inteligência Competitiva - AICOM
Instituto de Tecnologia em Imunobiológicos - Bio-Manguinhos
Fundação Oswaldo Cruz - Fiocruz

Pedro Henrique Monteiro Torres
Instituto de Biofísica Carlos Chagas Filho - IBCCF
Universidade Federal do Rio de Janeiro - UFRJ





Como máquinas “aprendem”?





Aprendizagem de Máquina

- Aprendizagem de máquina (amplamente também chamado de Aprendizado de Máquina) - *Machine Learning*
 - Disciplina que estuda como um computador pode aprender sem ser explicitamente programado. (Arthur Samuel, 1959)
- Aprendizagem no contexto computacional é adaptação!
 - Os programas de computador melhoram o seu desempenho em uma determinada tarefa a partir da coleta de dados.



Aprendizagem de Máquina

Aprendizagem de máquina

Supervisionada

Aprende com exemplos rotulados

Não supervisionada

Encontra padrões sem rótulos

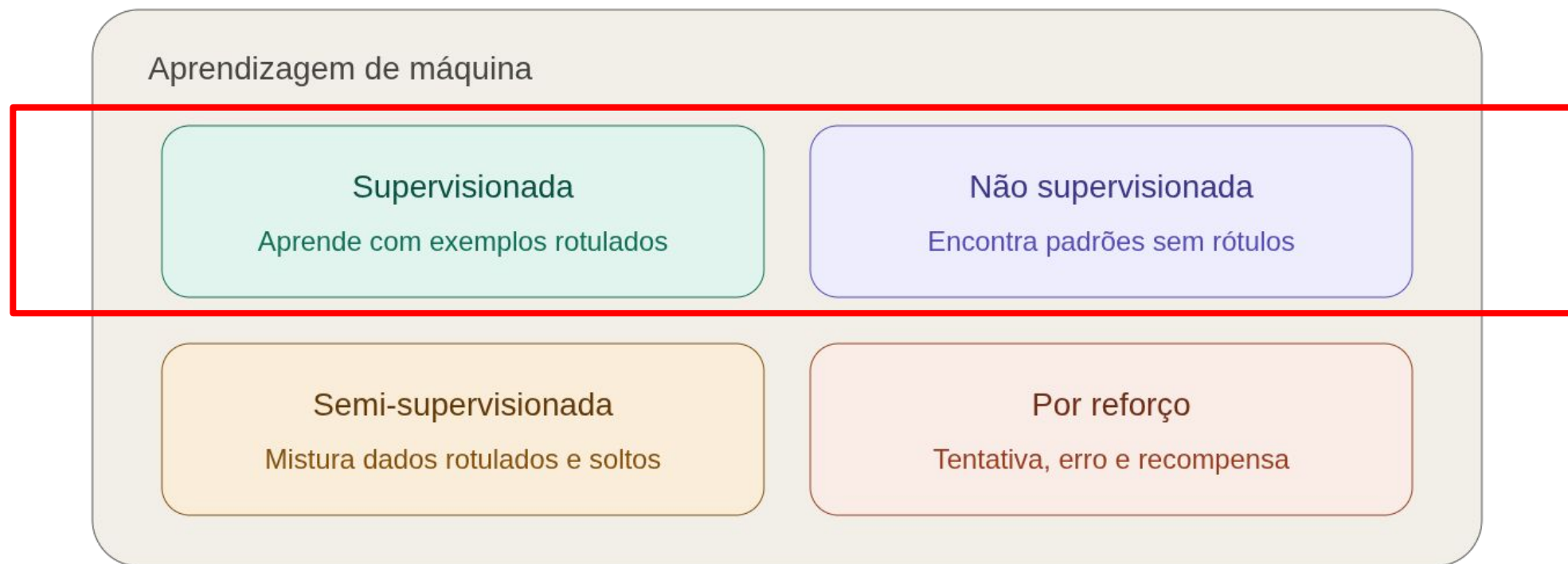
Semi-supervisionada

Mistura dados rotulados e soltos

Por reforço

Tentativa, erro e recompensa

Aprendizagem de Máquina



Aprendizagem Supervisionada

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Copacabana	2	Não	...	41,96
Copacabana	1	Sim	...	49,15
Centro	2	Não	...	28,05
Barra da Tijuca	3	Sim	...	30,86
...	...	...	...	...

Aprendizagem Supervisionada

tuplas, objetos, registros, amostras, ...

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Copacabana	2	Não	...	41,96
Copacabana	1	Sim	...	49,15
Centro	2	Não	...	28,05
Barra da Tijuca	3	Sim	...	30,86
...	...	...	...	...

alvo, classe, ...

atributos, variáveis, descritores, ...

Aprendizagem Supervisionada

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Copacabana	2	Não	...	41,96
Copacabana	1	Sim	...	49,15
Centro	2	Não	...	28,05
Barra da Tijuca	3	Sim	...	30,86
...	...	...	...	...

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Centro	1	Sim	...	?

Aprendizagem Supervisionada

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)	Alugo?
Copacabana	2	Não	...	41,96	Sim
Copacabana	1	Sim	...	49,15	Não
Centro	2	Não	...	28,05	Não
Barra da Tijuca	3	Sim	...	30,86	Sim
...	...	...	...	...	...

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Centro	1	Sim	...	?

Aprendizagem Supervisionada

Regressão

Classificação

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)	Alugo?
Copacabana	2	Não	...	41,96	Sim
Copacabana	1	Sim	...	49,15	Não
Centro	2	Não	...	28,05	Não
Barra da Tijuca	3	Sim	...	30,86	Sim
...	...	...	...	...	...

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)
Centro	1	Sim	...	?

Aprendizagem Supervisionada

Localização	Quartos	Garagem	...	Aluguel (R\$/m ²)	Alugo?
Copacabana	2	Não	...	41,96	Sim
Copacabana	1	Sim	...	49,15	Não
Centro	2	Não	...	28,05	Não
Barra da Tijuca	3	Sim	...	30,86	Sim
...	...	...	...	...	...

X

y

Aprendizagem Supervisionada

Title	ALogP	ALogp2	AMR	BCUTp-1	BCUTp-1h	fragC	apol	naAromAt	nAtom
Acebutolol	-2.1004	4.41168	67.0727	4.75506	9.76616	2152.06	55.7582	6	52
Amoxicillin	-1.874	3.51188	63.5964	4.78119	12.1043	1516.09	51.0391	6	44
Bendroflumethazide	-0.8075	0.65206	39.6471	4.24449	12.8541	1147.12	49.7141	12	41
Benzocaine	-0.4883	0.23844	20.5043	4.54796	8.52863	397.03	25.8787	6	23
Benzthlazine	-1.3398	1.79506	50.9651	6.69282	12.9721	1114.11	53.1231	10	40
Clozapine	0.0092	8.46E-05	45.0577	5.92358	11.5903	1519.05	50.9291	12	42
Dibucaine	-0.5602	0.31382	57.7981	5.09923	10.5046	2425.05	59.441	10	54
Diethylstilbestrol	-0.2302	0.05299	33.7086	4.85094	11.5338	1301.02	46.6199	12	40
Diflunisal	0.2593	0.06724	11.5289	4.03982	9.84339	423.05	31.7343	12	26
Dipyridamole	-3.3772	11.4055	95.8664	4.30728	11.5724	4981.12	80.9197	10	76
Folic_Acid	-3.3182	11.0105	52.3395	3.83215	9.33155	1817.13	58.6211	12	51
Furosemide	-0.9949	0.98983	33.6078	4.6696	12.2524	669.09	39.7447	11	32
Hydrochlorothiazide	-1.5436	2.3827	35.4327	7.26833	12.8018	404.1	32.1423	6	25
Imipramine	0.091	0.00828	39.2562	5.70631	11.5894	1789.02	51.643	12	45
Indomethacin	0.2922	0.08538	37.8739	4.12177	11.086	1249.06	50.5967	12	41
Ketoprofen	-0.2088	0.0436	21.9194	4.29006	9.91827	814.03	39.9011	12	33
Lidocaine	1.166	1.35956	45.8867	5.09304	9.98654	1249.03	42.3114	6	39
Meclofenamic_acid	1.4704	2.16208	27.2526	4.4468	11.3886	619.05	39.0387	12	30

Machine learning methods in chemoinformatics. Mitchell, J. B. O.; 2014

el (R\$/m ²)	Alugo?
1,96	Sim
9,15	Não
8,05	Não
0,86	Sim
...	...

y

Aprendizagem Supervisionada

- Componentes da aprendizagem
 - Entrada: $\mathbf{X}$
 - Saída: $\mathbf{y}$
 - Função alvo:
 - $f: \mathbf{X} \rightarrow \mathbf{y}$
 - Dados: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
 - ↓ ↓ ↓
 - Hipótese:
 - $g: \mathbf{X} \rightarrow \mathbf{y}$

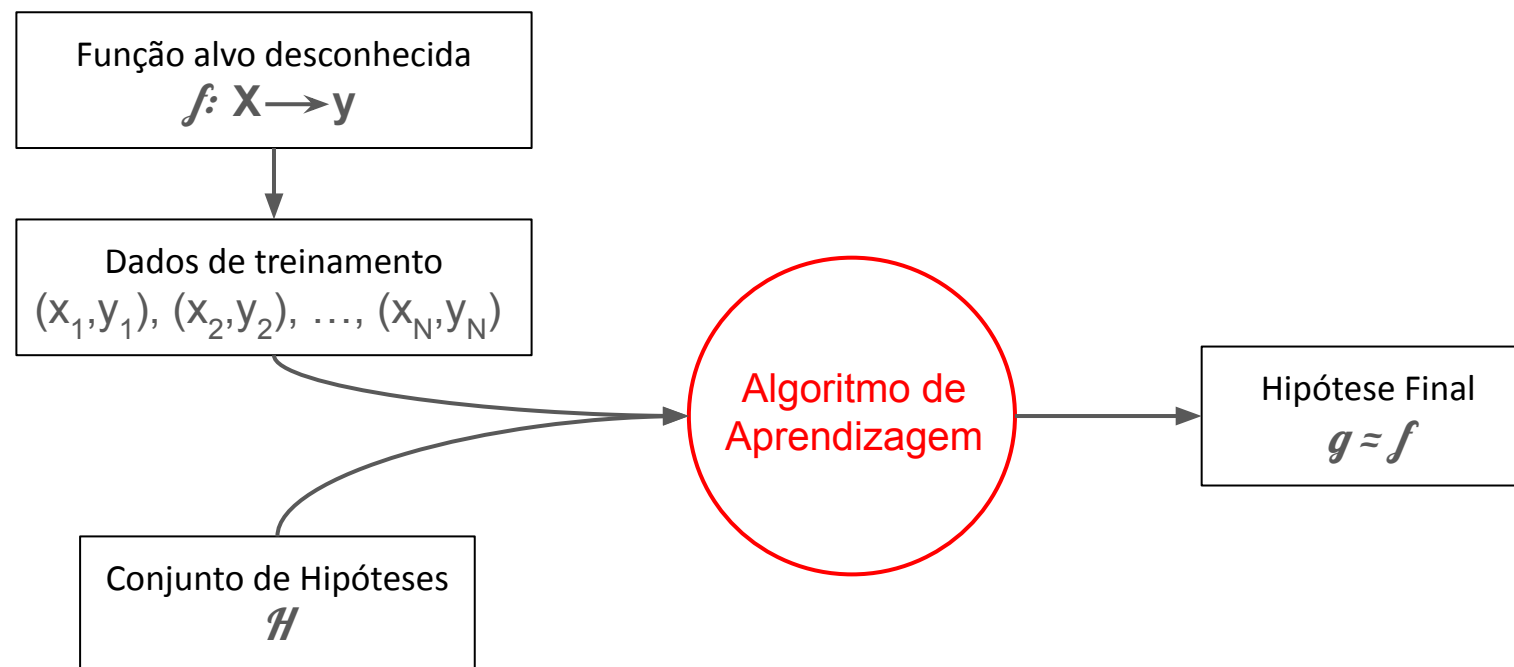


Aprendizagem Supervisionada

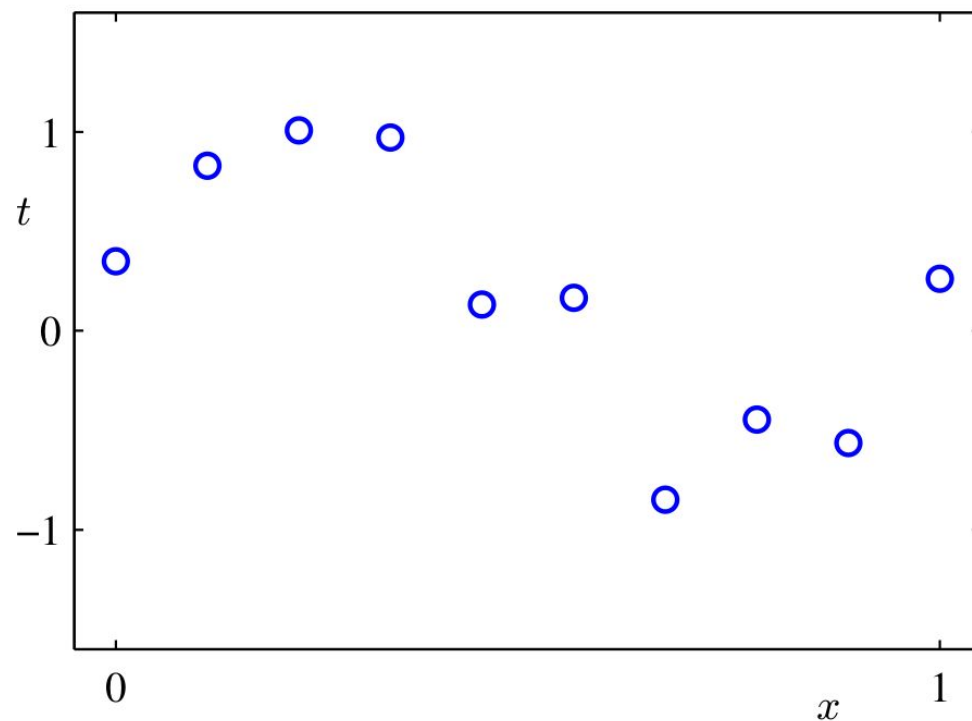
- Componentes da aprendizagem
 - Conjunto de hipóteses: $\mathcal{H}$
 - $g \in \mathcal{H}$
 - O Algoritmo de Aprendizagem: $\mathcal{A}$
 - Juntos formam o modelo de aprendizagem

Aprendizagem Supervisionada

- Componentes da aprendizagem



Aprendizagem Supervisionada





Aprendizagem Supervisionada

- Como encontrar um modelo a partir desses dados?
- Uma possibilidade é a obtenção de um modelo paramétrico
- Por exemplo, um polinômio:
 - $h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_M x^M$

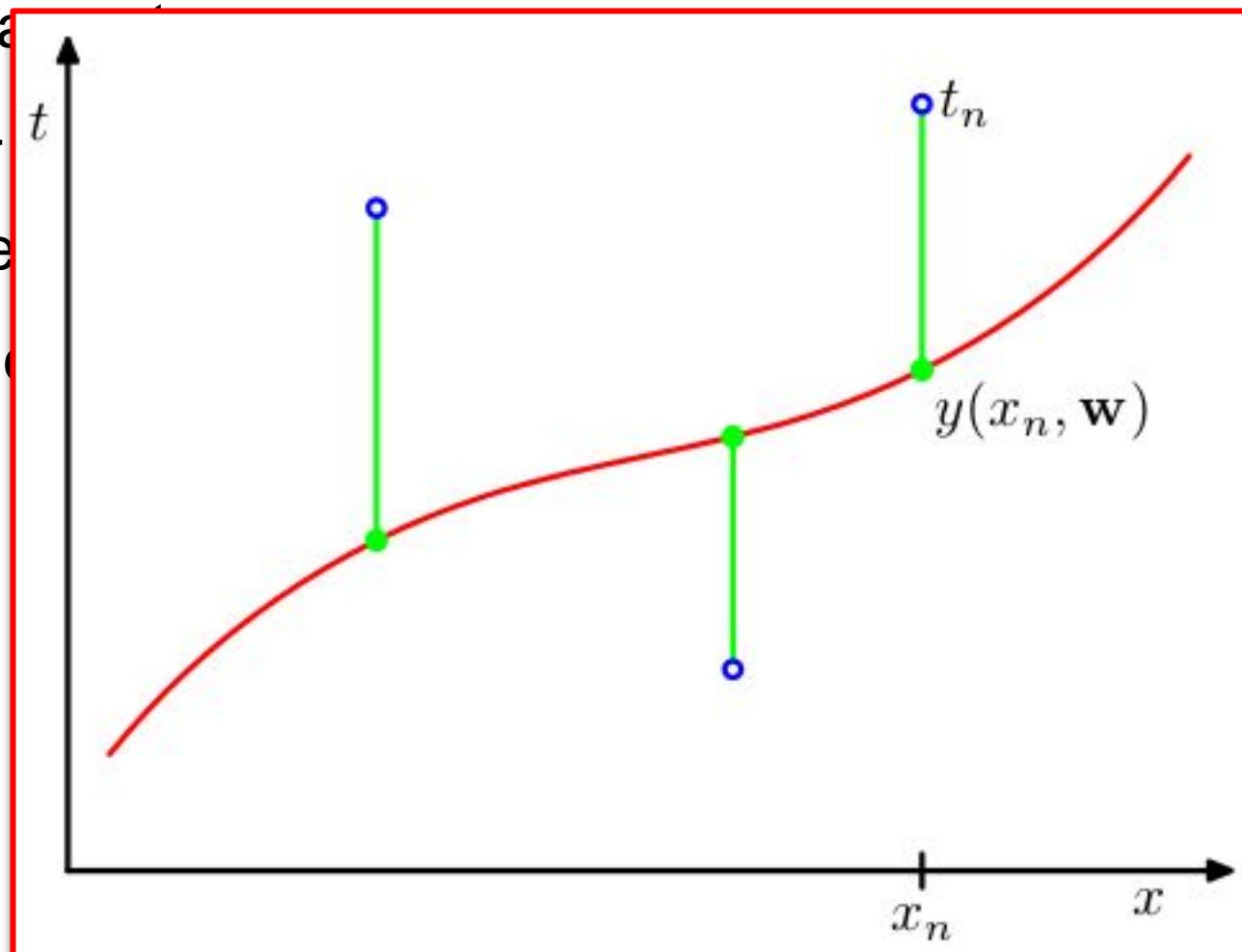


Aprendizagem Supervisionada

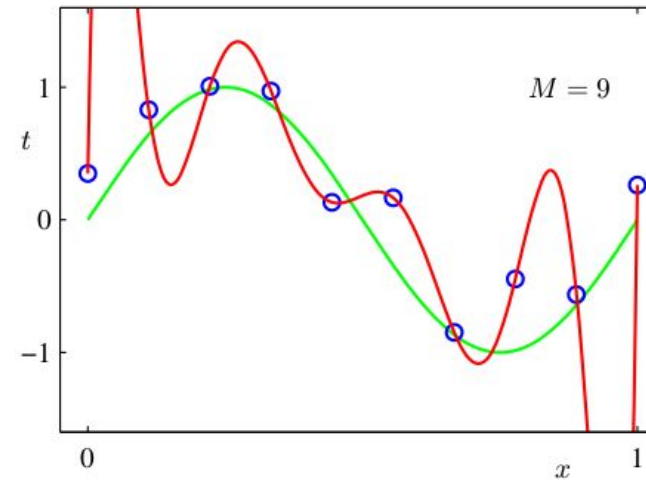
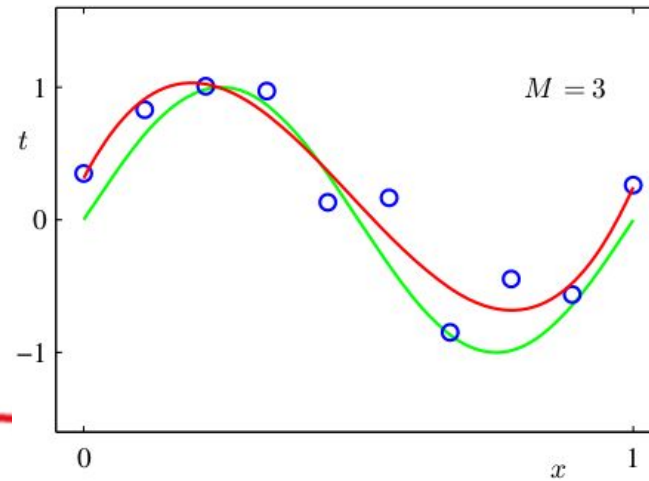
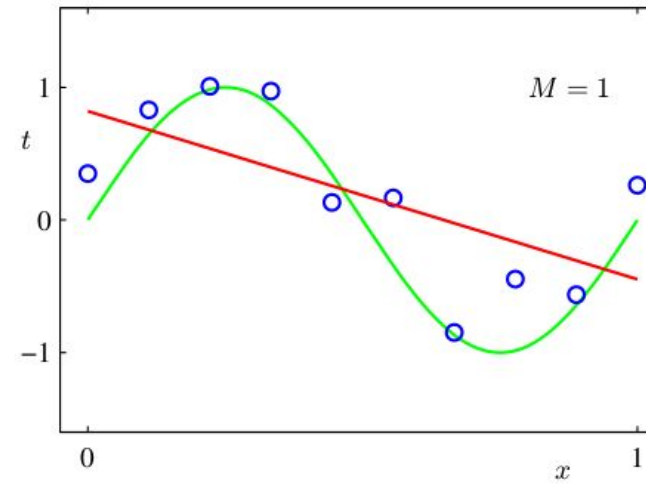
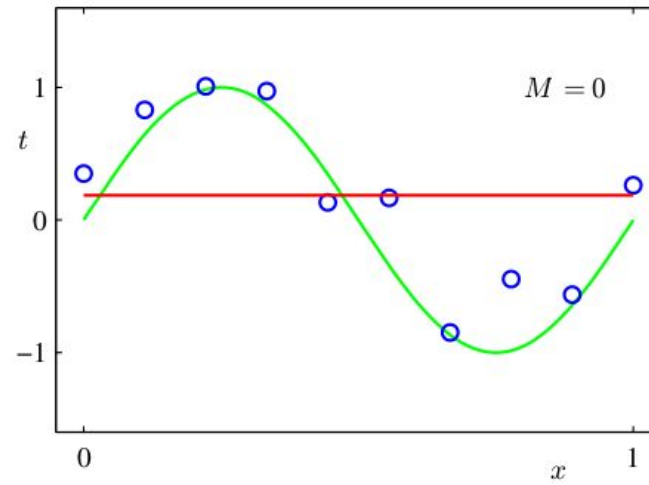
- Dado um conjunto de treinamento
 - $\{(x_i, y_i): i = 1, 2, \dots, m\}$
- O objetivo é encontrar θ de forma que $h_{\theta}(x_i) \approx y_i$
- Ou seja, minimize uma função de erro, por exemplo:
 - $J(\theta) = \sum_{i=1} (y_i - h_{\theta}(x_i))^2$

Aprendizagem Supervisionada

- Dado um conjunto de treinamento
 - $\{(x_i, y_i): i = 1, 2, \dots, m\}$
- O objetivo é encontrar θ de
- Ou seja, minimize uma função
 - $J(\theta) = \sum_{i=1}^m (y_i - h_{\theta}(x_i))^2$

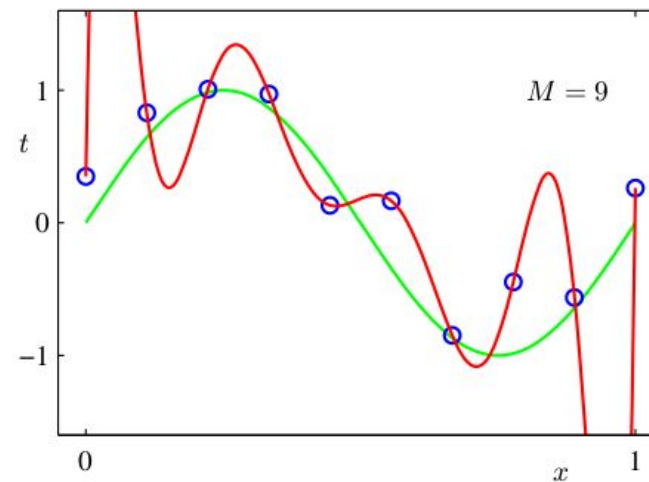
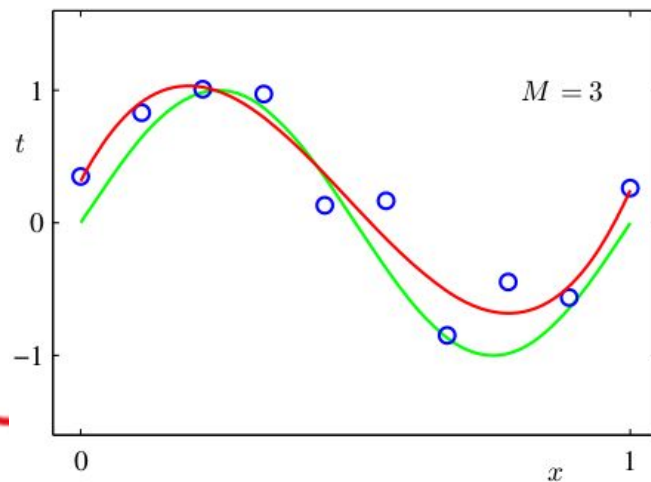
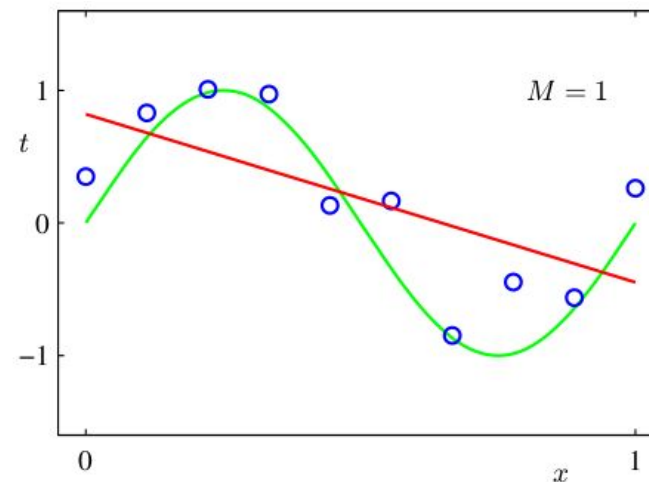
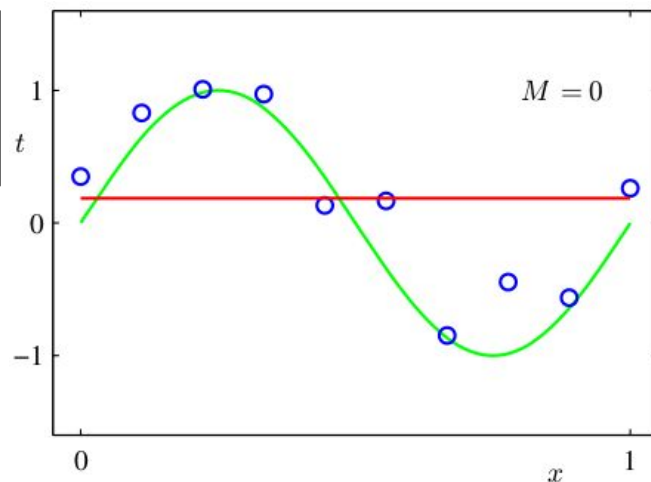


Aprendizagem Supervisionada



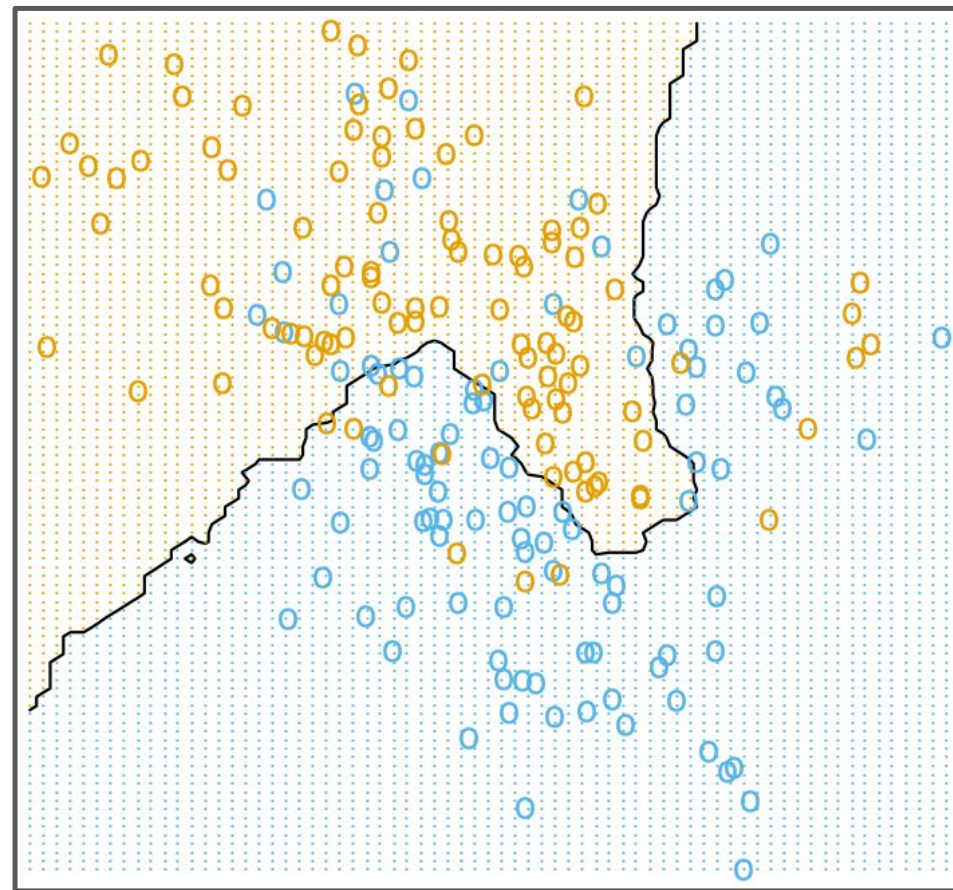
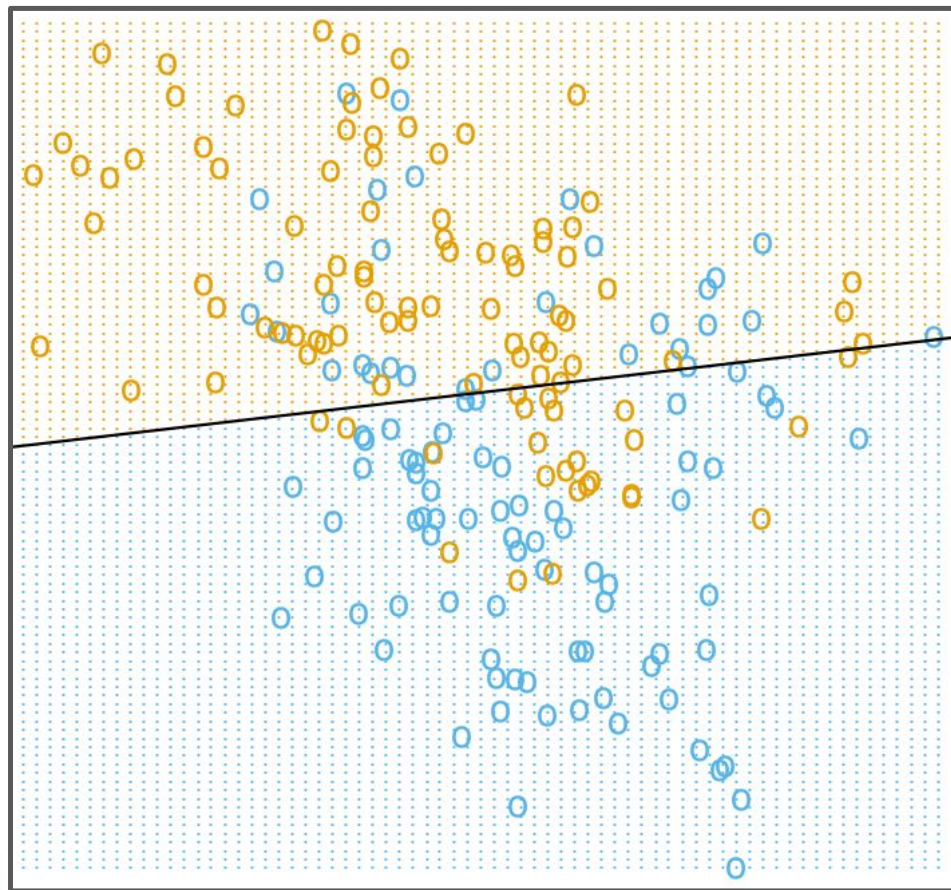
Aprendizagem Supervisionada

“underfitting”
- bias (viés) alto

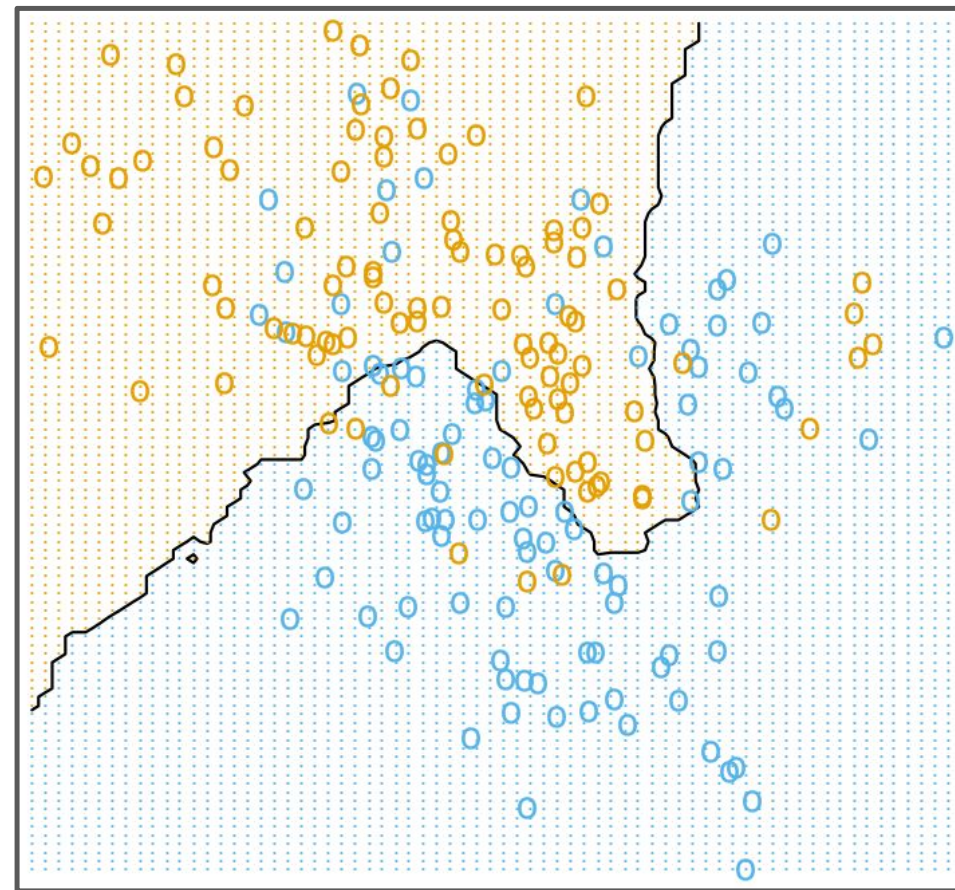
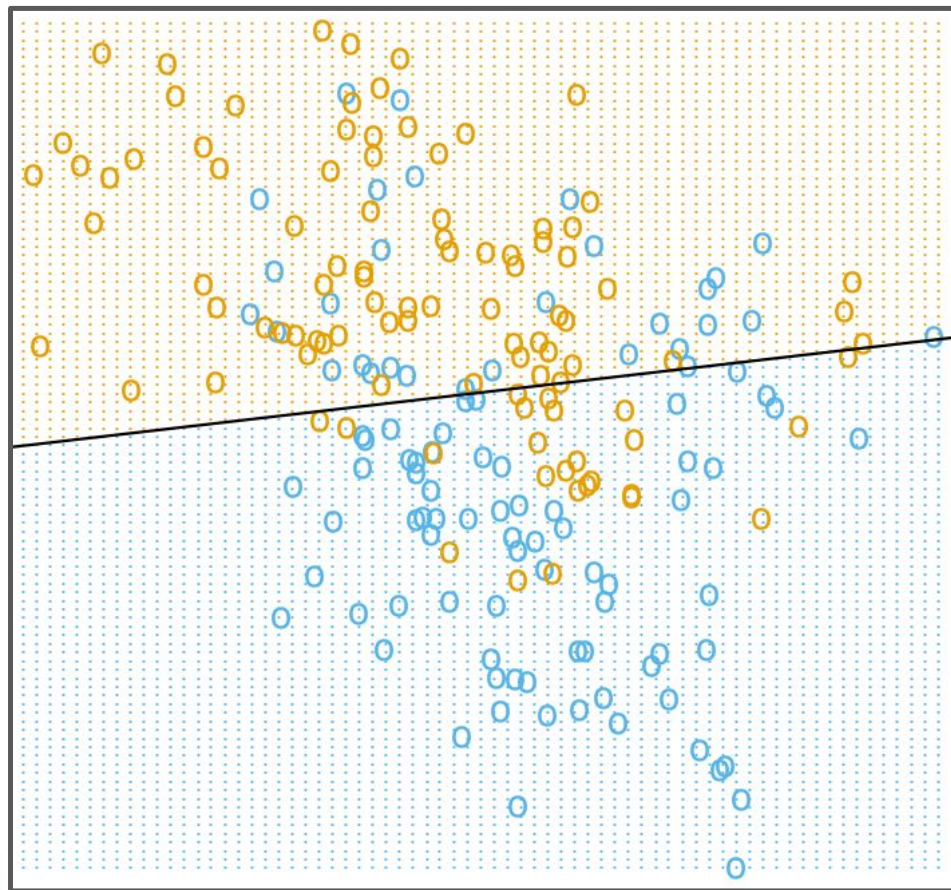


“overfitting”
- variância alta

Aprendizagem Supervisionada



Aprendizagem Supervisionada



o equivalente para a classificação!



Aprendizagem Supervisionada

- Qual o objetivo da aprendizagem supervisionada?
- Como escolher entre vários modelos?
- Deve-se buscar modelos que “passem” por todos os pontos conhecidos?





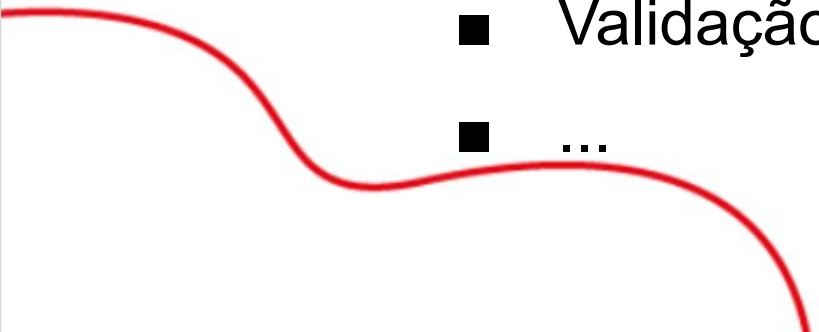
Aprendizagem Supervisionada

- Quais as possíveis origens de ruídos nos dados?
 - Erros na coleta (qualidade de dados)
 - Erros na manipulação dos dados (digitação, arredondamentos,...)
 - Atributos insuficientes para descrever o fenômeno



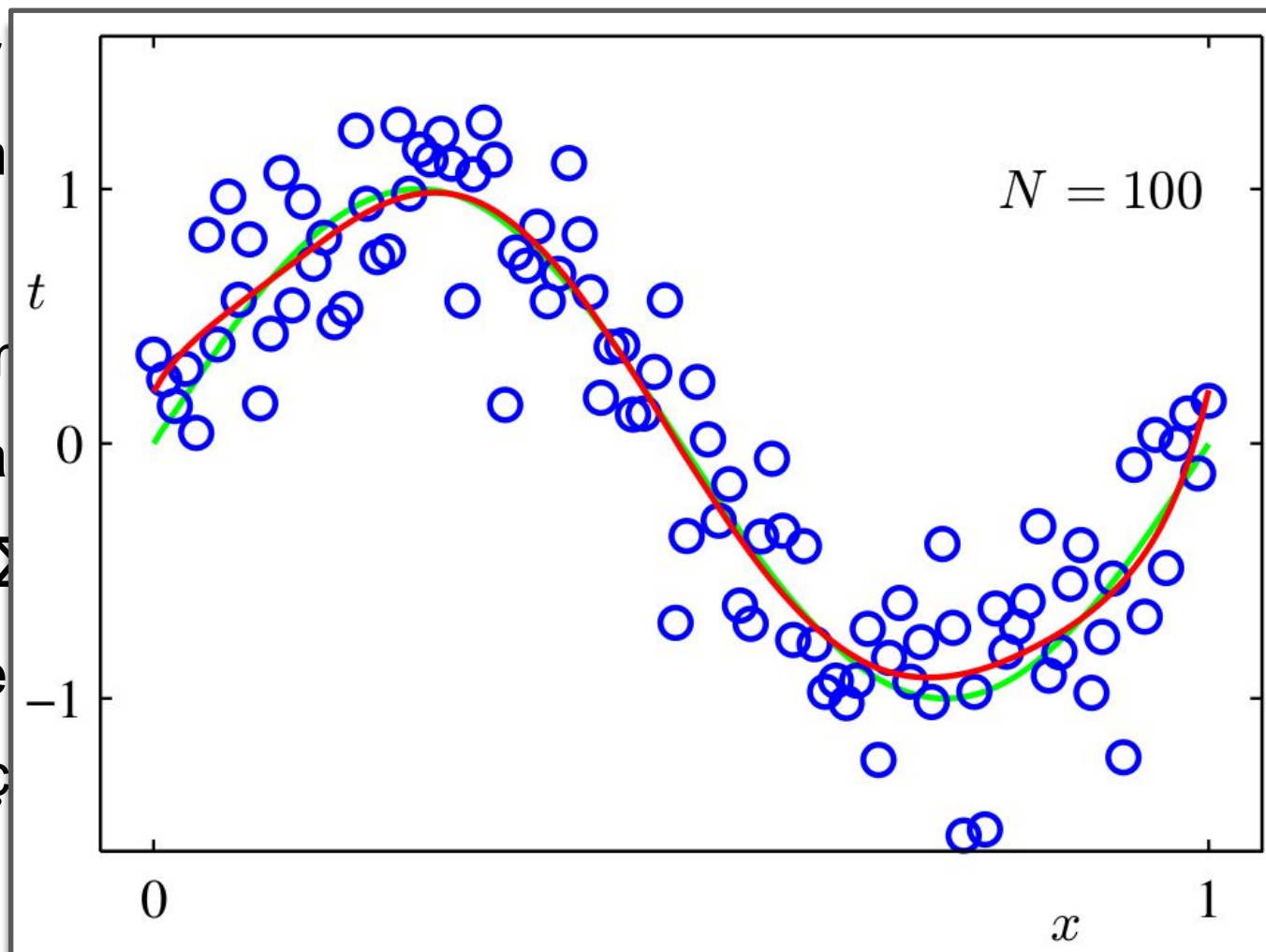


Aprendizagem Supervisionada

- Como equilibrar o viés e a variâncias?
 - Coletar o máximo de dados possíveis
 - Adotar um modelo complexo e evitar o excesso de ajuste:
 - Regularização
$$J(\theta) = \sum_{i=1} (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1} \theta_i$$
 - Comitês de modelo
 - Validação cruzada
 - ...
- 
- 

Aprendizagem Supervisionada

- Como equilibrar
 - Coletar o m
 - Adotar um r
 - Regula
 - $J(\theta) = \sum$
 - Comitê
 - Validaç
 - ...





Seleção de Modelos

- Acurácia (generalidade)
 - Simplicidade
 - Interpretabilidade
- 
- 

Seleção de Modelos



Seleção de Modelos

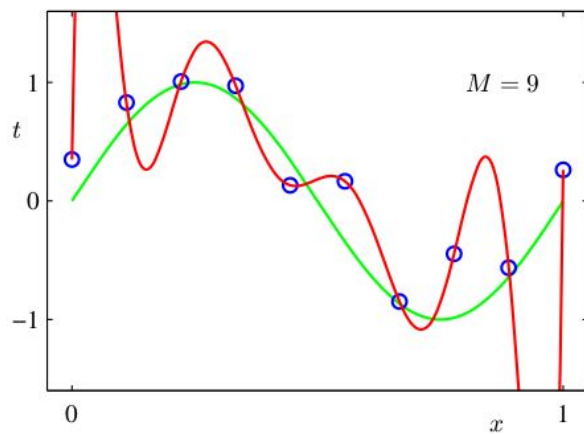
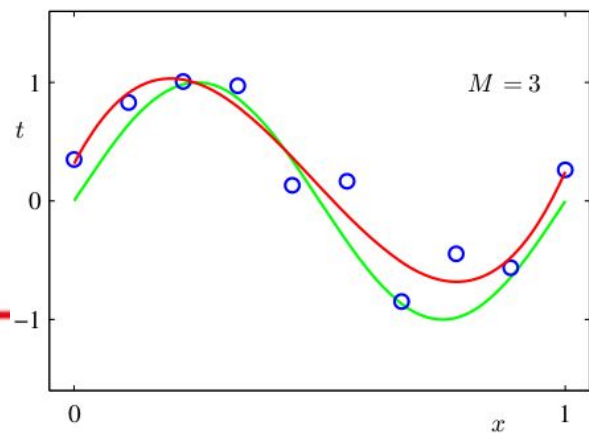
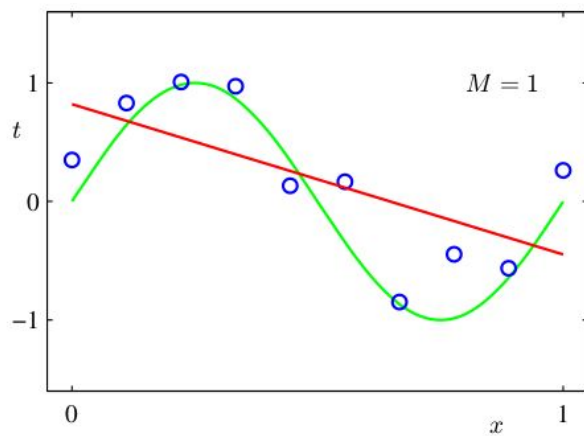
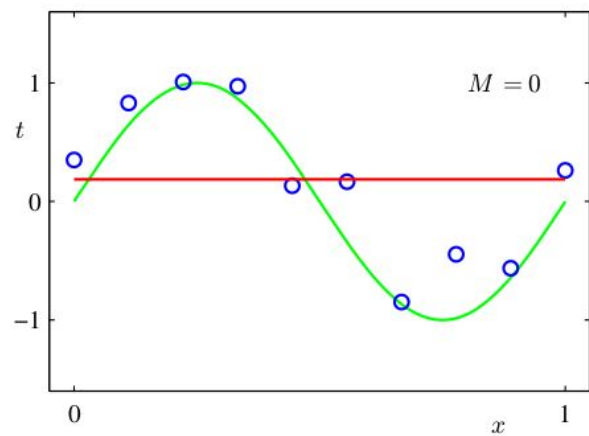


Single Dataset

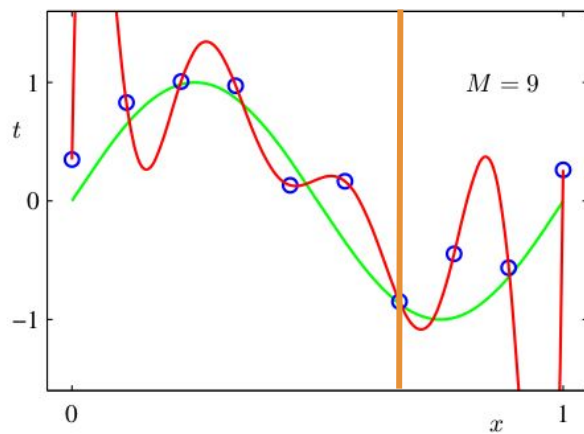
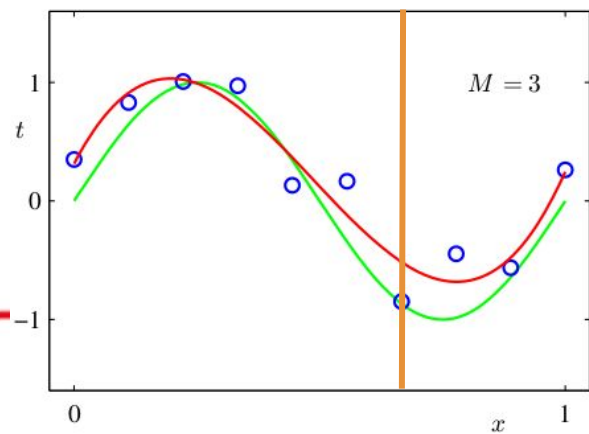
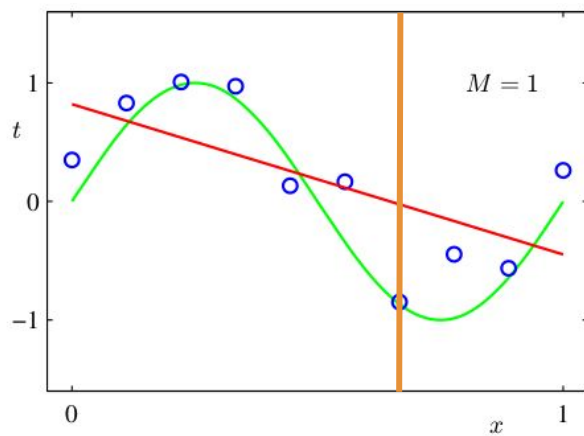
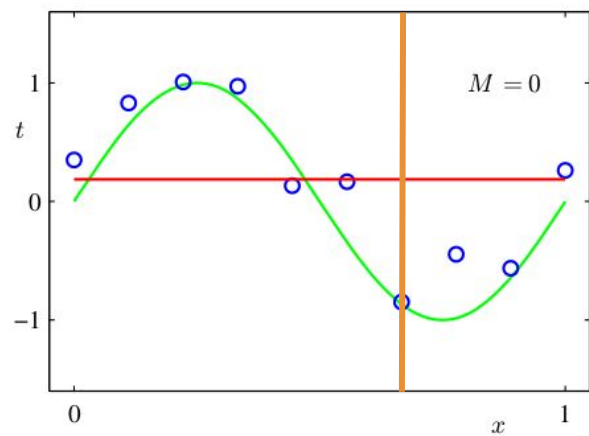


Single Dataset

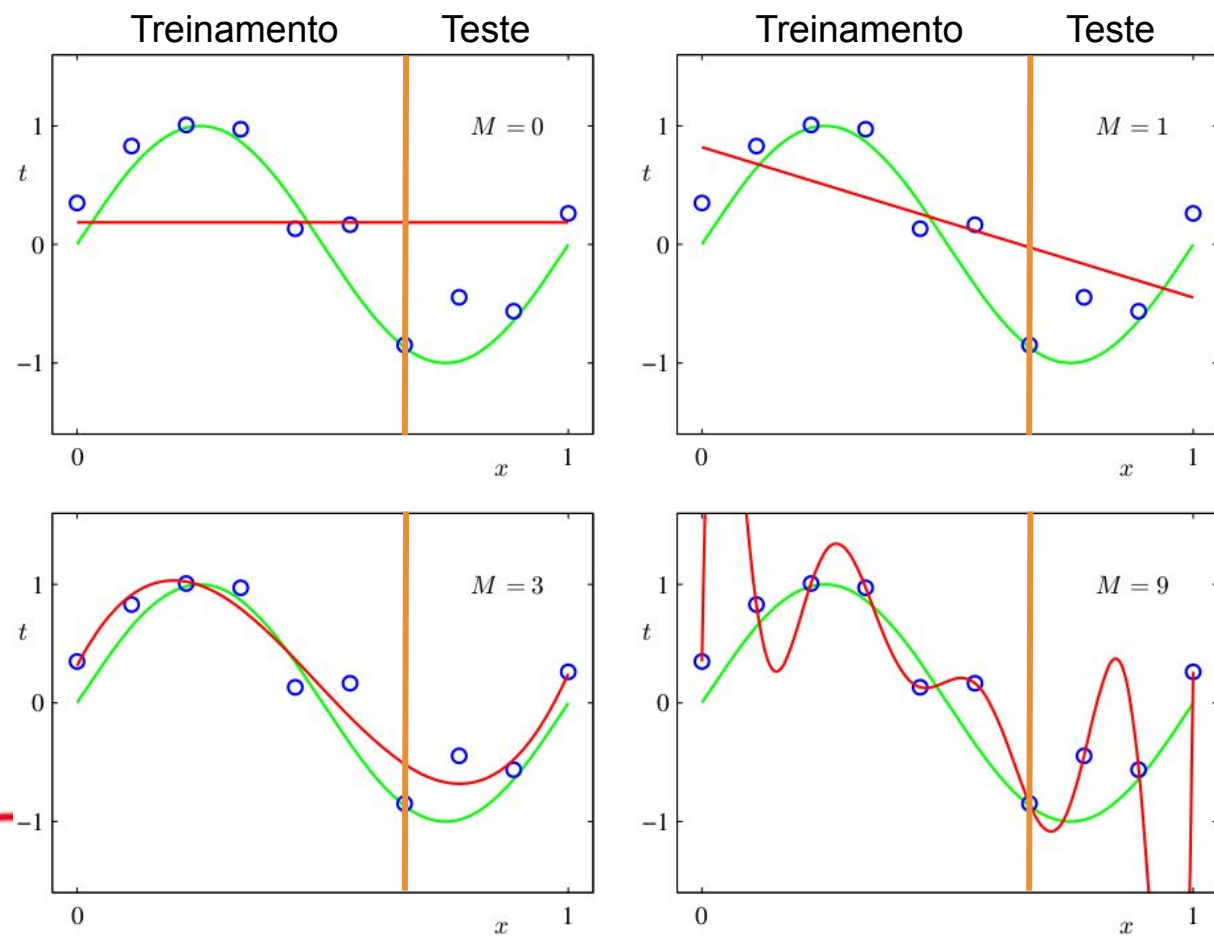
Seleção de Modelos



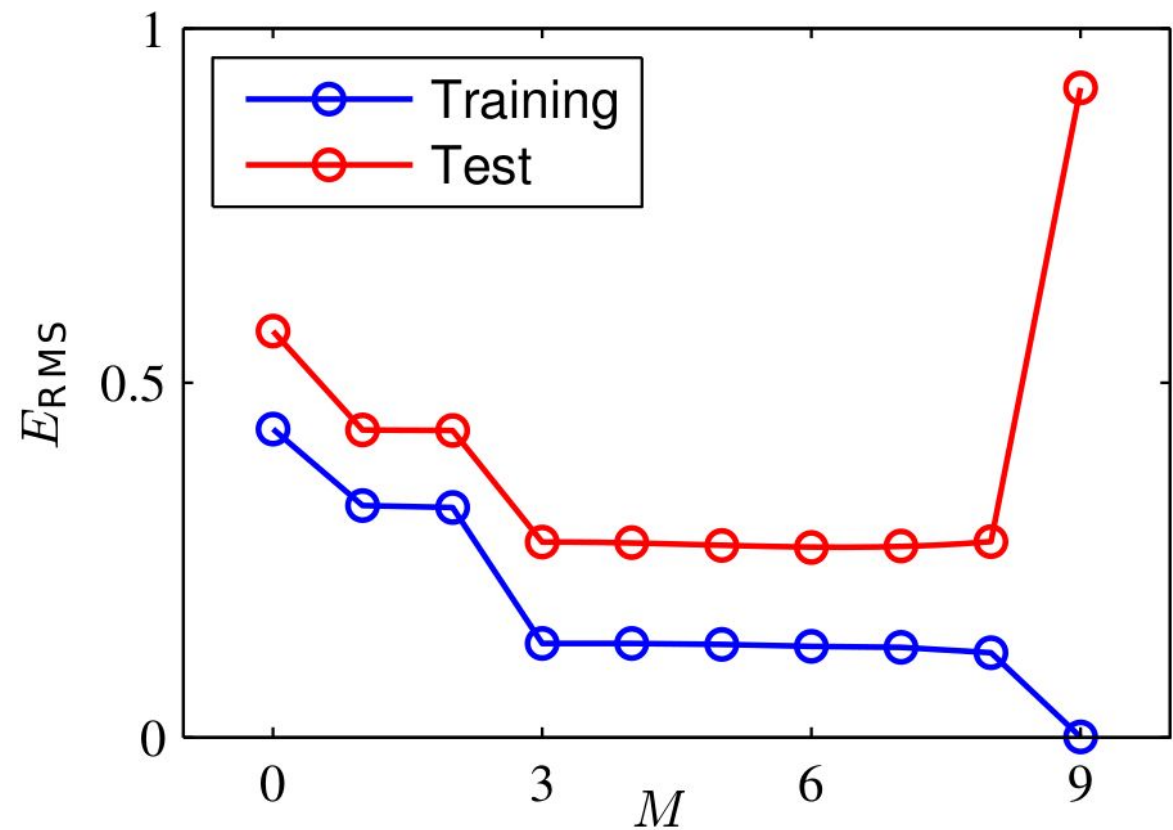
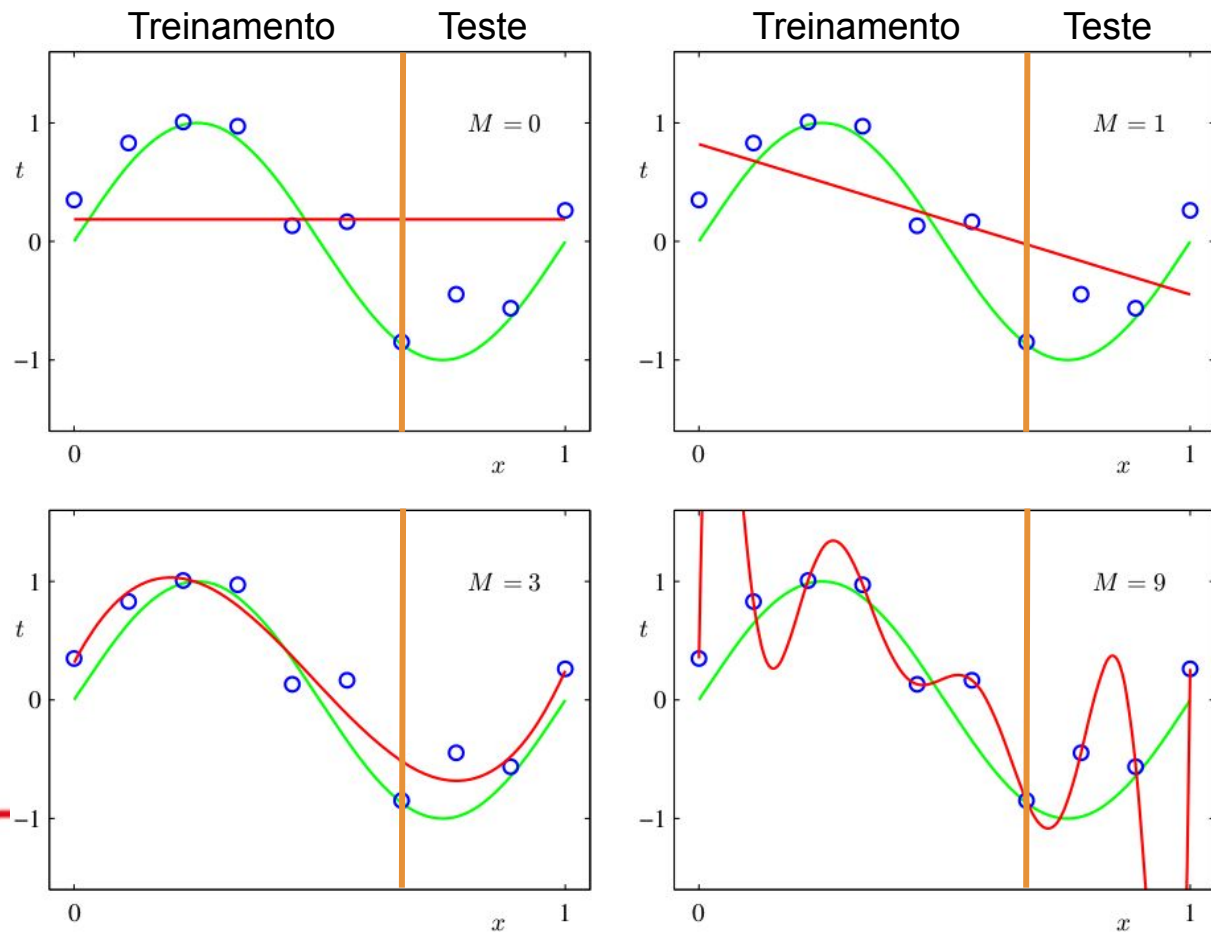
Seleção de Modelos



Seleção de Modelos



Seleção de Modelos

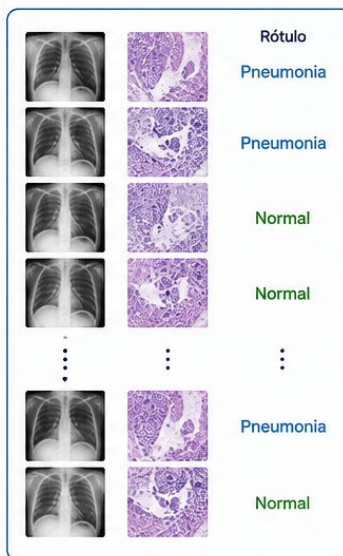


Aprendizagem Supervisionada

Uso do conjunto de teste para avaliação de um método de Aprendizagem de Máquina

1. Dados disponíveis

Base de dados rotulada



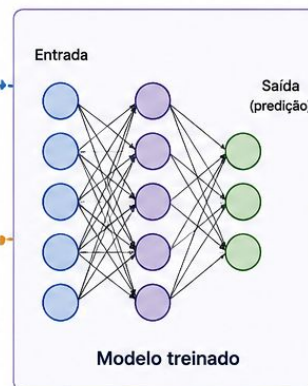
2. Divisão dos dados

Particionamento



3. Treinamento do modelo

Aprendizado a partir do conjunto de treinamento



4. Avaliação no conjunto de teste

Avaliação FINAL do desempenho do modelo em dados nunca vistos durante o treinamento

Predições no conjunto de teste			
Verdadeiro (rótulo real)		Predito pelo modelo	Correto?
	Pneumonia	Pneumonia	✓
	Normal	Normal	✓
	Pneumonia	Normal	✗
	Normal	Pneumonia	✗
...	...	...	...

Métricas de desempenho (exemplos)

Matriz de Confusão

	Predito	
	Pneumonia	Normal
Real Pneumonia	85	15
Real Normal	10	90

Acurácia: 87,5%

Precisão (Pneumonia): 89,5%

Revocação (Pneumonia): 85,0%

F1-score (Pneumonia): 87,2%

...

Por que usar um conjunto de teste separado?



- ✓ Fornece uma estimativa imparcial da capacidade de generalização do modelo.
- ✓ Evita o overfitting ao conjunto de treinamento/validação.
- ✓ Representa o desempenho esperado em dados novos e não vistos.

- Usado para aprender o modelo
- - - Usado para ajustar o modelo
- Usado apenas para avaliação final

Regra de ouro



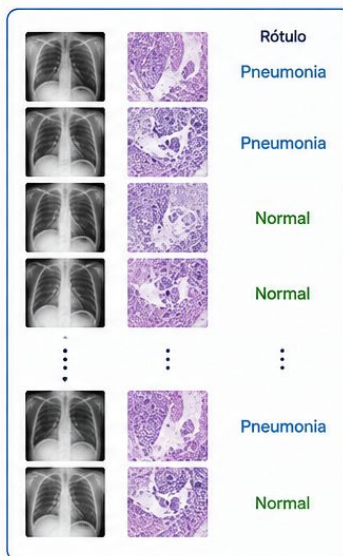
O conjunto de teste deve ser usado APENAS uma vez, no final.

Aprendizagem Supervisionada

Uso do conjunto de teste para avaliação de um método de Aprendizagem de Máquina

1. Dados disponíveis

Base de dados rotulada



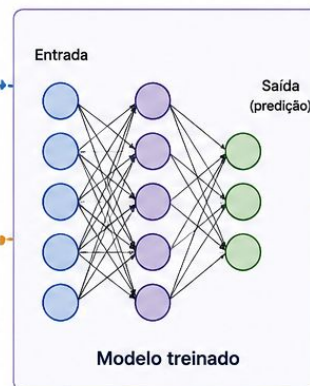
2. Divisão dos dados

Particionamento



3. Treinamento do modelo

Aprendizado a partir do conjunto de treinamento



4. Avaliação no conjunto de teste

Avaliação FINAL do desempenho do modelo em dados nunca vistos durante o treinamento

Predições no conjunto de teste			
Verdadeiro (rótulo real)		Predito pelo modelo	Correto?
	Pneumonia	Pneumonia	✓
	Normal	Normal	✓
	Pneumonia	Normal	✗
	Normal	Pneumonia	✗
...	...	...	...

Métricas de desempenho (exemplos)

Matriz de Confusão

	Predito	
	Pneumonia	Normal
Real Pneumonia	85	15
Real Normal	10	90

Acurácia: 87,5%

Precisão (Pneumonia): 89,5%

Revocação (Pneumonia): 85,0%

F1-score (Pneumonia): 87,2%

...

Por que usar um conjunto de teste separado?



- ✓ Fornece uma estimativa imparcial da capacidade de generalização do modelo.
- ✓ Evita o overfitting ao conjunto de treinamento/validação.
- ✓ Representa o desempenho esperado em dados novos e não vistos.

- Usado para aprender o modelo
- - - Usado para ajustar o modelo
- Usado apenas para avaliação final

Regra de ouro



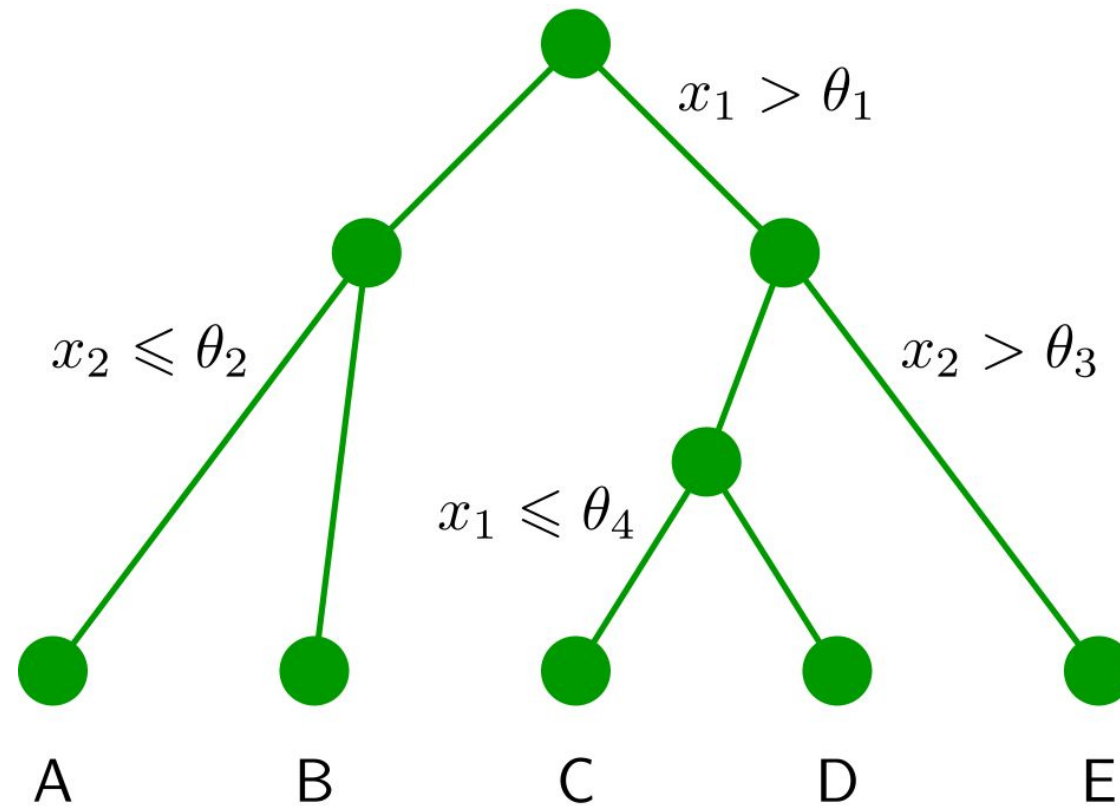
O conjunto de teste deve ser usado APENAS uma vez, no final.



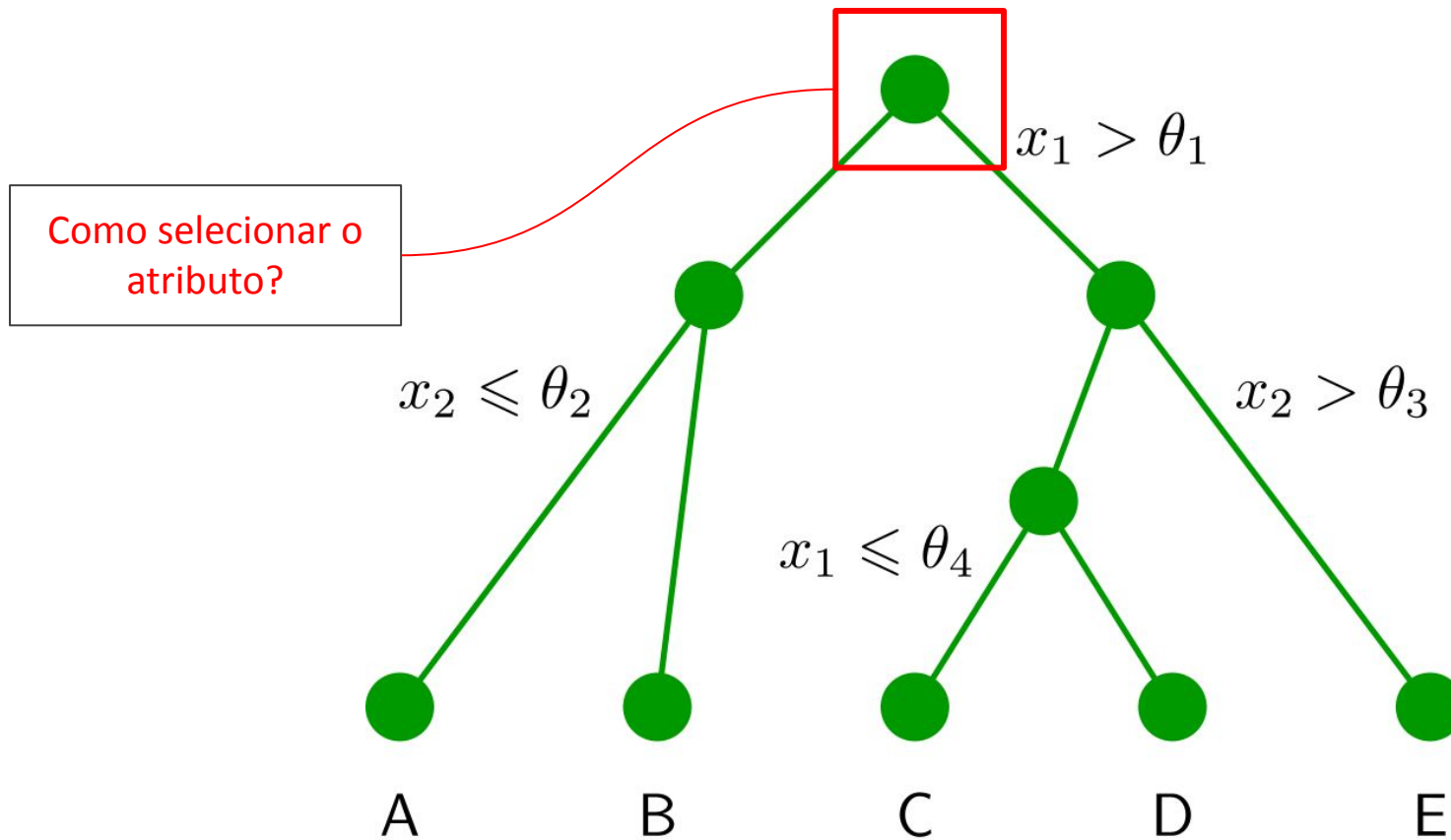
Árvores de Decisão

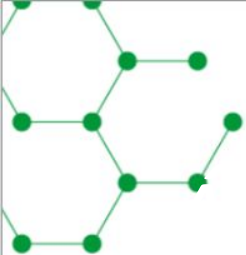


Árvores de Decisão



Árvores de Decisão





Como selecionar o atributo?

V1	V2	Classe
0.1	0.3	A
0.2	0.01	B
0.18	0.1	B
0.28	0.32	A
0.15	0.4	A

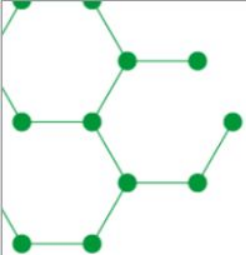
V1	V2	Classe
0.1	0.3	A
0.18	0.1	B
0.15	0.4	A

Sim

$V1 < 0.2$

Não

V1	V2	Classe
0.2	0.01	B
0.28	0.32	A



Como selecionar o atributo?

V1	V2	Classe
0.1	0.3	A
0.2	0.01	B
0.18	0.1	B
0.28	0.32	A
0.15	0.4	A

$V2 < 0.2$

Sim

Não

V1	V2	Classe
0.2	0.01	B
0.18	0.1	B

V1	V2	Classe
0.1	0.3	A
0.28	0.32	A
0.15	0.4	A

Árvores de Decisão

Entropia

$$H(S) = -\sum p_i \log_2 p_i$$

Gini

$$\text{Gini}(S) = 1 - \sum p_i^2$$

Redução de Variância

$$\text{Var}(S) = (1/n) \sum (y_i - \bar{y})^2$$

Erro de Classificação

$$\text{Error}(S) = 1 - \max(p_i)$$

Árvores de Decisão

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from matplotlib import pyplot as plt
from sklearn import tree

datafile = "iris.csv"
df = pd.read_csv(datafile, header = 0)
display(df)

feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
X = df[feature_cols]
y = df.species

# 70% treinamento e 30% para teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

clf = DecisionTreeClassifier()

# Train Decision Tree Classifier
clf = clf.fit(X_train, y_train)

# Predict the response for test dataset
y_pred = clf.predict(X_test)

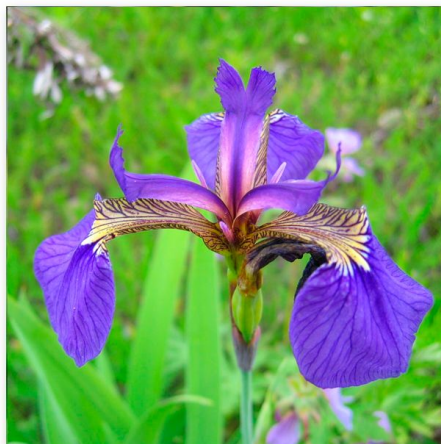
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

fig, axes = plt.subplots(nrows = 1, ncols = 1, figsize=(10, 10))
tree.plot_tree(clf, filled=True)
plt.show()
```

Base de dados das Flores de Íris

Iris flower dataset

Setosa



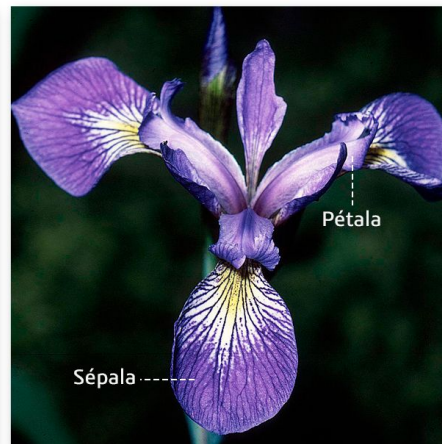
Ирис шестилепестный (Кавказка), Dominio público, via Wikimedia Commons

Versicolor



Charles de Mille-Isles from Mille-Isles, Canada, CC BY 2.0, via Wikimedia Commons

Virginica



Robert H. Mohlenbrock. Courtesy of USDA NRCS, Public domain, via Wikimedia Commons

gini = 0.0
samples = 1
value = [0, 1, 0]

gini = 0.0
samples = 3
value = [0, 0, 3]

gini = 0.0
samples = 3
value = [0, 0, 3]

gini = 0.0
samples = 1
value = [0, 1, 0]

$x[2] \leq 4.85$
gini = 0.056
samples = 35
value = [0, 1, 34]

$x[2] \leq 3.1$
gini = 0.375
samples = 4
value = [0, 1, 3]

gini = 0.0
samples = 31
value = [0, 0, 31]

$x[3] \leq 0.8$

Árvores de Decisão

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier
from sklearn.model_selection import train_test_split # Import train_test_split function
from sklearn import metrics
from matplotlib import pyplot as plt
from sklearn import tree

datafile = "iris.csv"
df = pd.read_csv(datafile, header = 0)
display(df)

feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
X = df[feature_cols]
y = df.species

# 70% treinamento e 30% para teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

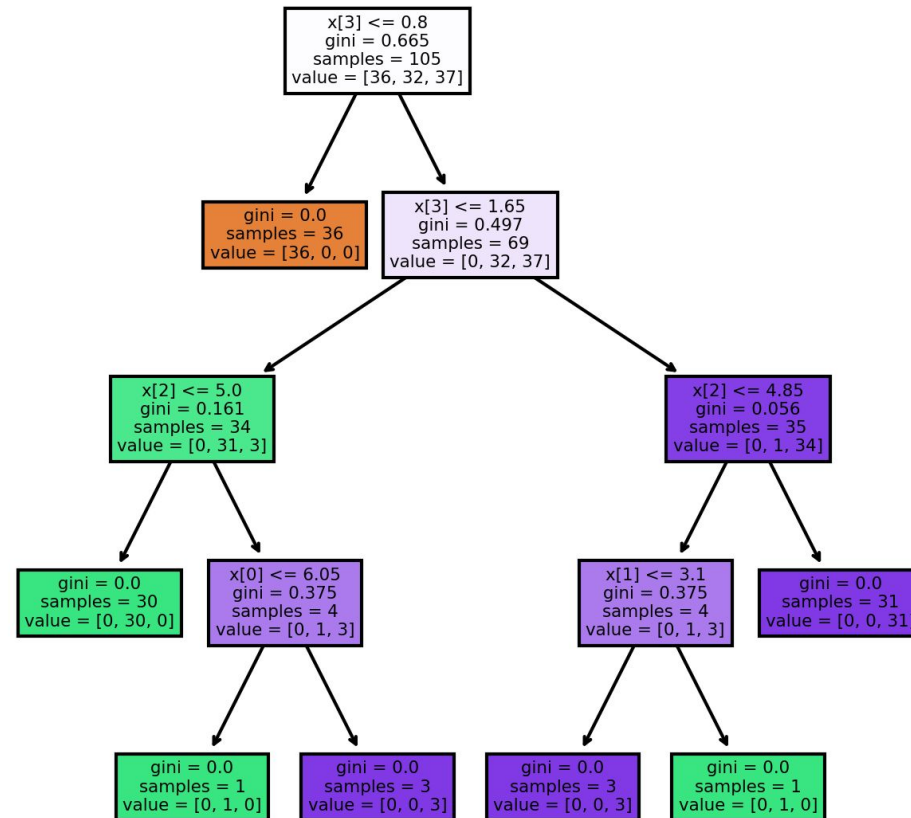
clf = DecisionTreeClassifier()

# Train Decision Tree Classifier
clf = clf.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test)

print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (5,5), dpi=300)
tree.plot_tree(clf, filled=True)
plt.show()
```



Árvores de Decisão

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier # Import Decision Tree Classifier
from sklearn.model_selection import train_test_split # Import train_test_split function
from sklearn import metrics
from matplotlib import pyplot as plt
from sklearn import tree

datafile = "iris.csv"
df = pd.read_csv(datafile, header = 0)
display(df)

feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
X = df[feature_cols]
y = df.species

# 70% treinamento e 30% para teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

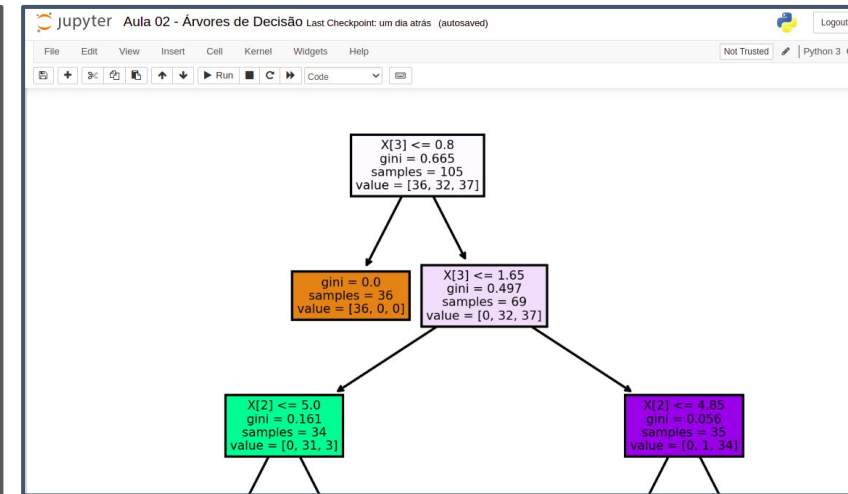
clf = DecisionTreeClassifier()

# Train Decision Tree Classifier
clf = clf.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test)

print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

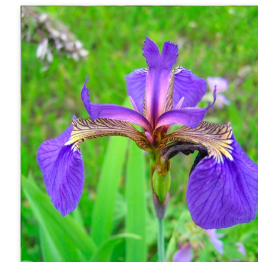
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (5,5), dpi=300)
tree.plot_tree(clf, filled=True)
plt.show()
```



Base de dados das Flores de Íris

Iris flower dataset

Setosa



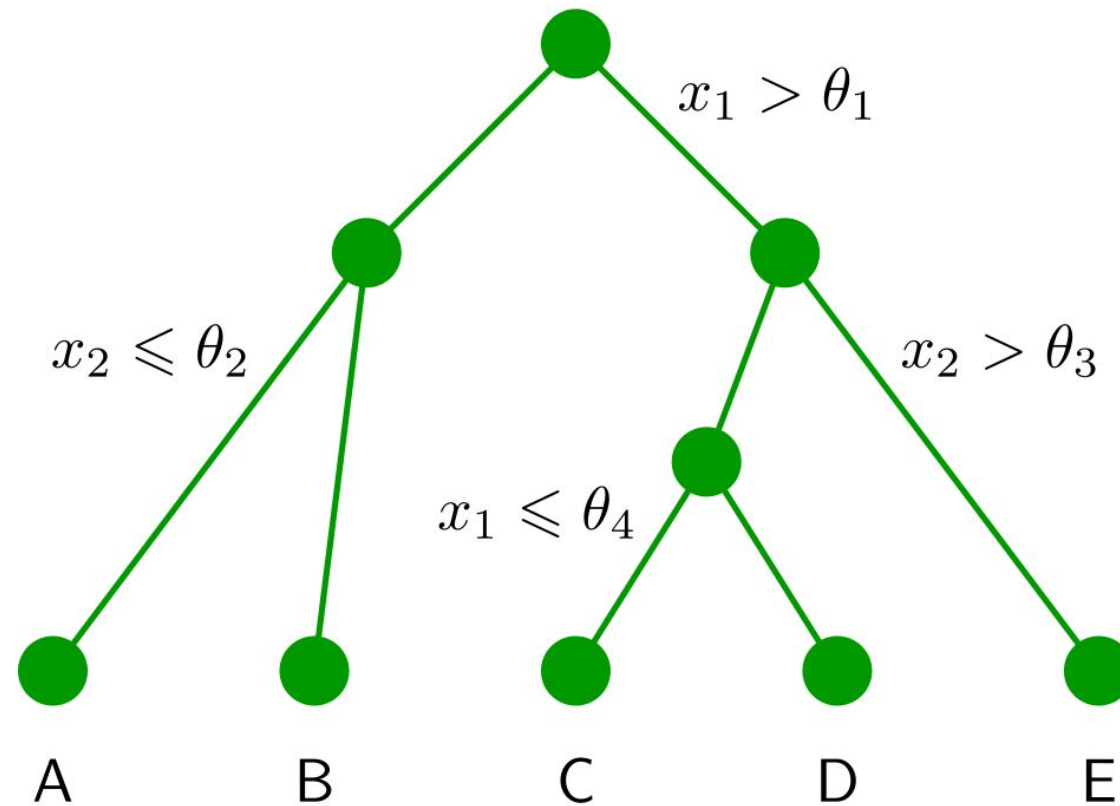
Versicolor



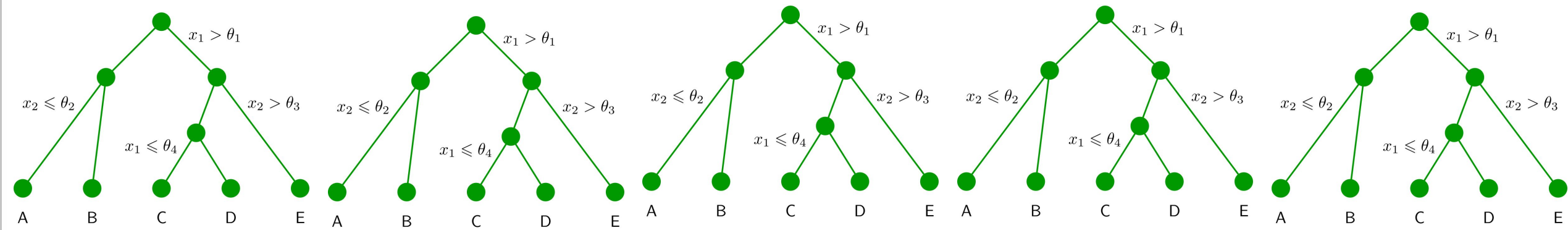
Virginica



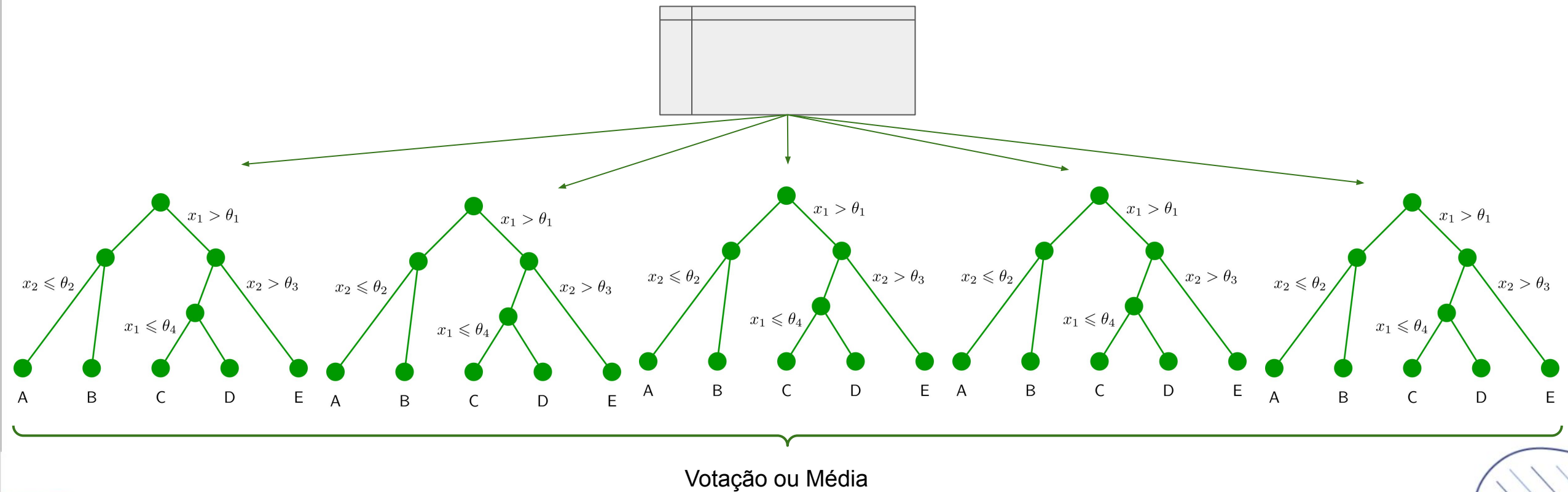
Árvores de Decisão



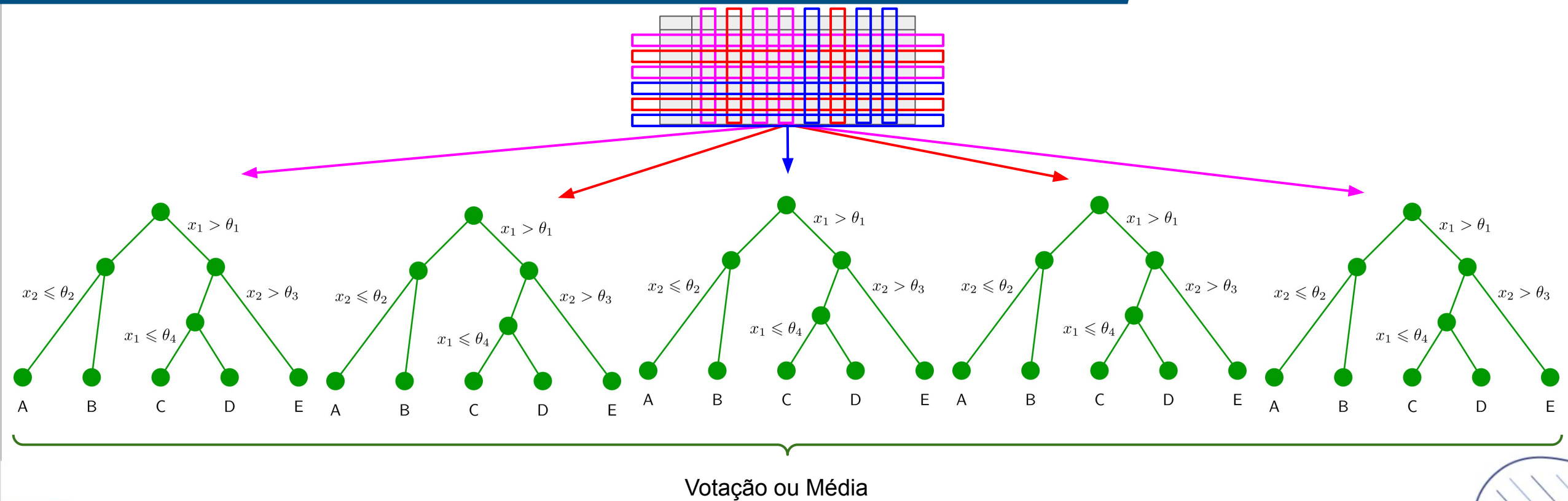
Aprendizagem em conjunto (Ensemble)



Aprendizagem em conjunto (Ensemble)



Floresta Aleatória (Random Forest)



Floresta Aleatória (Random Forest)

Parâmetro	Default	O que controla
<code>n_estimators</code>	<code>100</code>	Número de árvores na floresta
<code>criterion</code>	<code>'gini'</code>	Métrica de qualidade do split (<code>'gini'</code> , <code>'entropy'</code> , <code>'log_loss'</code>)
<code>max_depth</code>	<code>None</code>	Profundidade máxima de cada árvore (<code>None</code> = cresce até esgotar as folhas)
<code>min_samples_split</code>	<code>2</code>	Mínimo de amostras exigidas para dividir um nó interno
<code>min_samples_leaf</code>	<code>1</code>	Mínimo de amostras exigidas numa folha
<code>min_weight_fraction_leaf</code>	<code>0.0</code>	Igual ao anterior, mas como fração do total de amostras (com peso)
<code>max_features</code>	<code>'sqrt'</code>	Nº de atributos considerados em cada split (<code>'sqrt'</code> , <code>'log2'</code> , <code>int</code> , <code>float</code> ou <code>None</code>)
<code>max_leaf_nodes</code>	<code>None</code>	Limita o número total de folhas por árvore
<code>min_impurity_decrease</code>	<code>0.0</code>	Só divide um nó se a redução de impureza for > esse valor
<code>bootstrap</code>	<code>True</code>	Se cada árvore usa amostragem com reposição (bagging) do dataset
<code>oob_score</code>	<code>False</code>	Se calcula a acurácia usando as amostras "out-of-bag" (não usadas no bootstrap daquela árvore)
<code>n_jobs</code>	<code>None</code>	Nº de processos paralelos no treino/predição (<code>-1</code> = usa todos os núcleos)
<code>random_state</code>	<code>None</code>	Semente para reprodutibilidade
<code>verbose</code>	<code>0</code>	Nível de log durante o treino
<code>warm_start</code>	<code>False</code>	Se <code>True</code> , reaproveita árvores já treinadas e adiciona mais ao chamar <code>.fit()</code> de novo
<code>class_weight</code>	<code>None</code>	Pondera classes desbalanceadas (<code>'balanced'</code> , <code>dict</code> , ou lista de <code>dicts</code>)
<code>ccp_alpha</code>	<code>0.0</code>	Parâmetro de poda por complexidade de custo (cost-complexity pruning)
<code>max_samples</code>	<code>None</code>	Se <code>bootstrap=True</code> , controla quantas amostras (nº ou fração) cada árvore sorteia
<code>monotonic_cst</code>	<code>None</code>	Impõe restrições de monotonicidade entre atributo e predição (adição mais recente)

Floresta Aleatória (Random Forest)

```
In [8]: import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from matplotlib import pyplot as plt
from sklearn import tree
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, matthews_corrcoef
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
In [2]: datafile = "../iris.csv"
df = pd.read_csv(datafile, header = 0)
df.head()
```

```
Out[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [3]: feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
x = df[feature_cols]
y = df.species
```

```
In [4]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)
```

```
In [9]: classifier = RandomForestClassifier(n_estimators=100) #Random Forest
classifier = classifier.fit(x_train,y_train)
y_pred = classifier.predict(x_test)
```

```
In [10]: print("Accuracy:",accuracy_score(y_test, y_pred))

Accuracy: 0.9555555555555556
```

```
In [12]: #Importância dos atributos
feature_imp = pd.Series(classifier.feature_importances_,index=x.columns).sort_values(ascending=False)
feature_imp
```

```
Out[12]: petal_width      0.457216
petal_length    0.416301
sepal_length    0.093285
sepal_width     0.033199
dtype: float64
```

Floresta Aleatória (Random Forest)

```
In [8]: import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from matplotlib import pyplot as plt
from sklearn import tree
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, matthews_corrcoef
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
In [2]: datafile = "../iris.csv"
df = pd.read_csv(datafile, header = 0)
df.head()
```

```
Out[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [3]: feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
x = df[feature_cols]
y = df.species
```

```
In [4]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)
```

```
In [9]: classifier = RandomForestClassifier(n_estimators=100) #Random Forest
classifier = classifier.fit(x_train,y_train)
y_pred = classifier.predict(x_test)
```

```
In [10]: print("Accuracy:",accuracy_score(y_test, y_pred))

Accuracy: 0.9555555555555556
```

```
In [12]: #Importância dos atributos
feature_imp = pd.Series(classifier.feature_importances_,index=x.columns).sort_values(ascending=False)
feature_imp
```

```
Out[12]: petal_width    0.457216
petal_length    0.416301
sepal_length    0.093285
sepal_width     0.033199
dtype: float64
```

Floresta Aleatória (Random Forest)

```
In [8]: import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from matplotlib import pyplot as plt
from sklearn import tree
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, matthews_corrcoef
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
In [2]: datafile = "../iris.csv"
df = pd.read_csv(datafile, header = 0)
df.head()
```

```
Out[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [3]: feature_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
x = df[feature_cols]
y = df.species
```

```
In [4]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=1)
```

```
In [9]: classifier = RandomForestClassifier(n_estimators=100) #Random Forest
classifier = classifier.fit(x_train,y_train)
y_pred = classifier.predict(x_test)
```

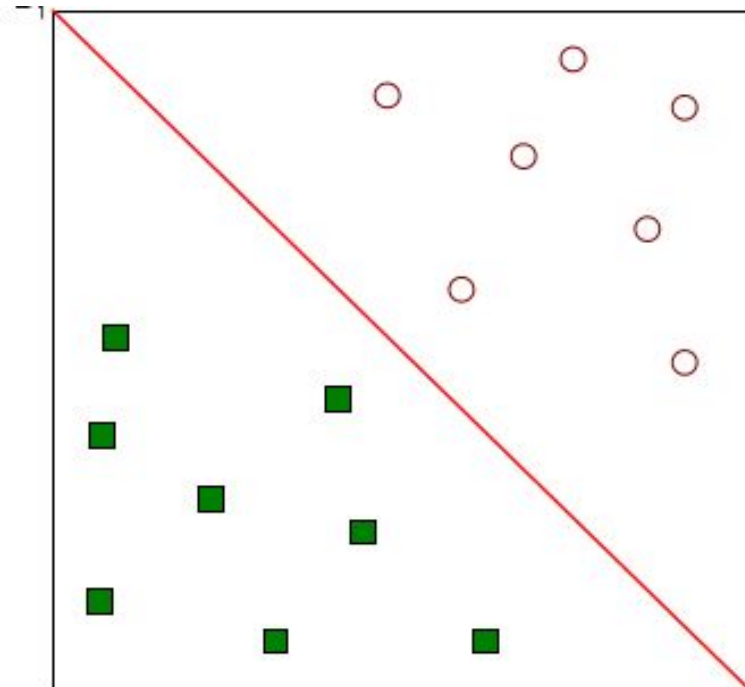
```
In [10]: print("Accuracy:",accuracy_score(y_test, y_pred))

Accuracy: 0.9555555555555556
```

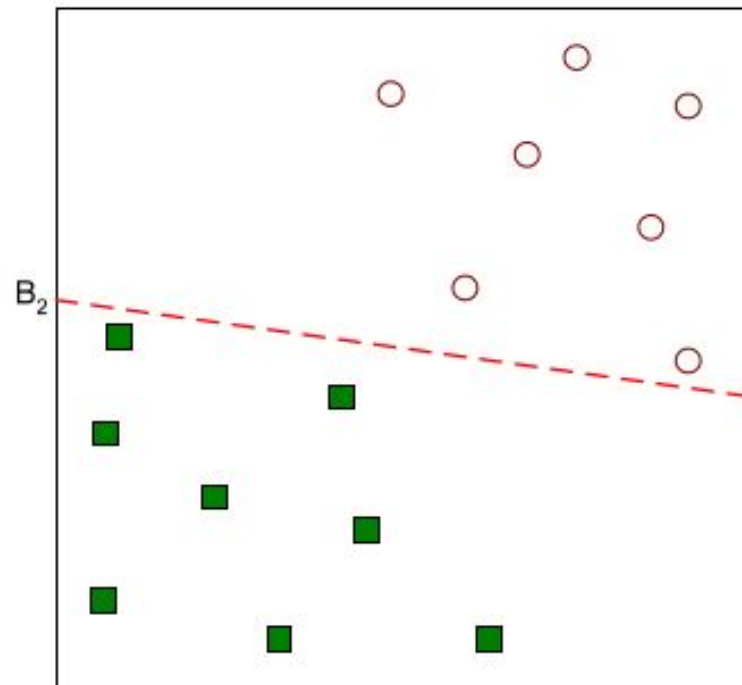
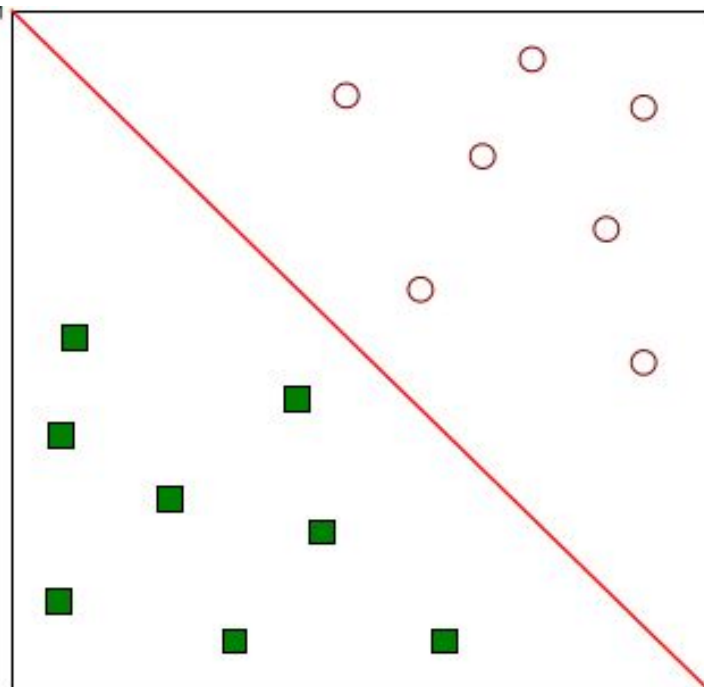
```
In [12]: #Importância dos atributos
feature_imp = pd.Series(classifier.feature_importances_,index=x.columns).sort_values(ascending=False)
feature_imp
```

```
Out[12]: petal_width    0.457216
petal_length    0.416301
sepal_length    0.093285
sepal_width     0.033199
dtype: float64
```

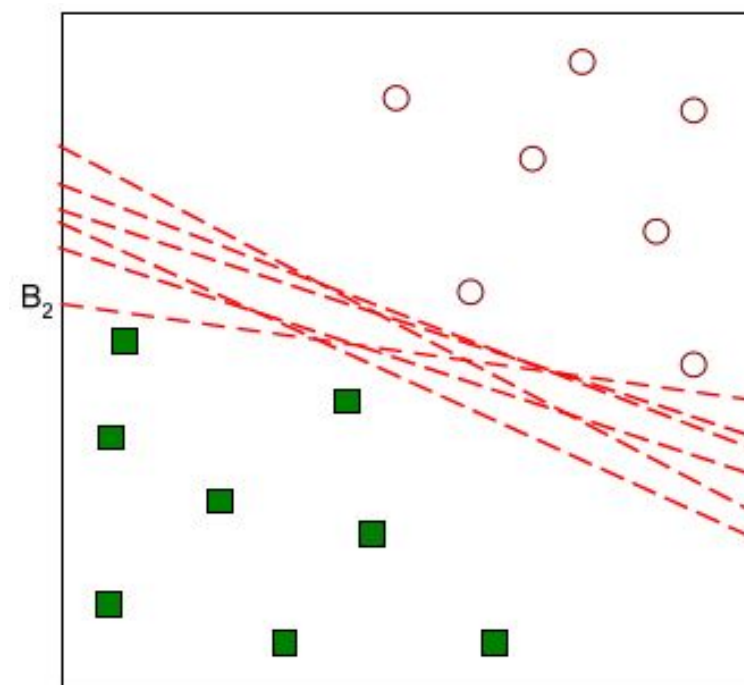
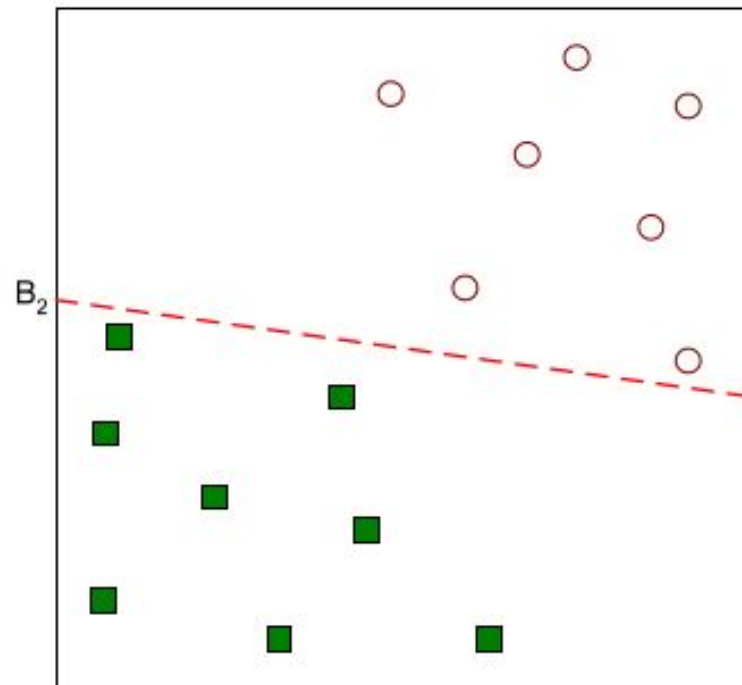
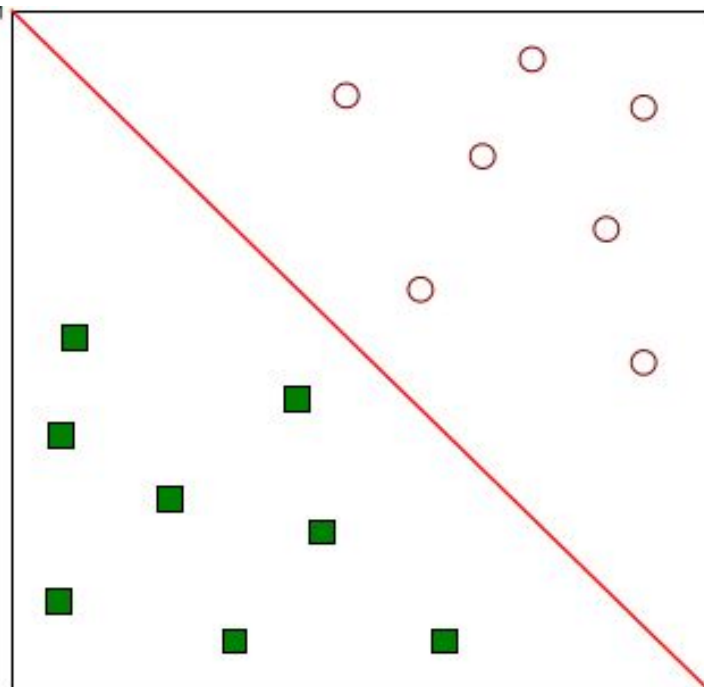
Máquina de Vetores Suporte - SVM



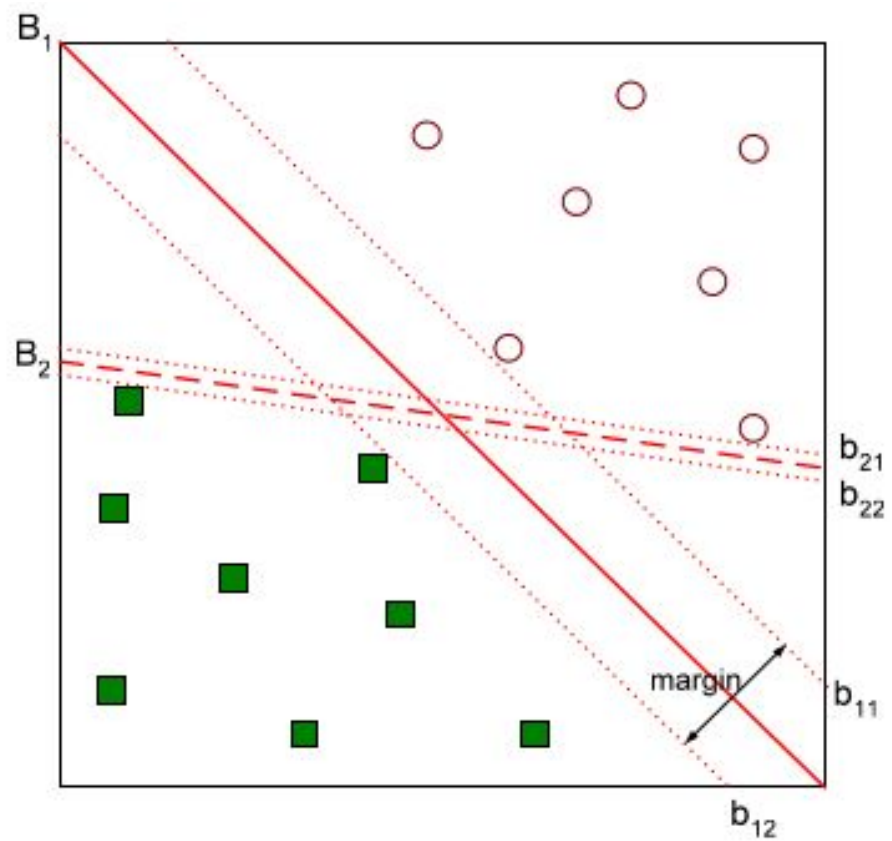
Máquina de Vetores Suporte - SVM



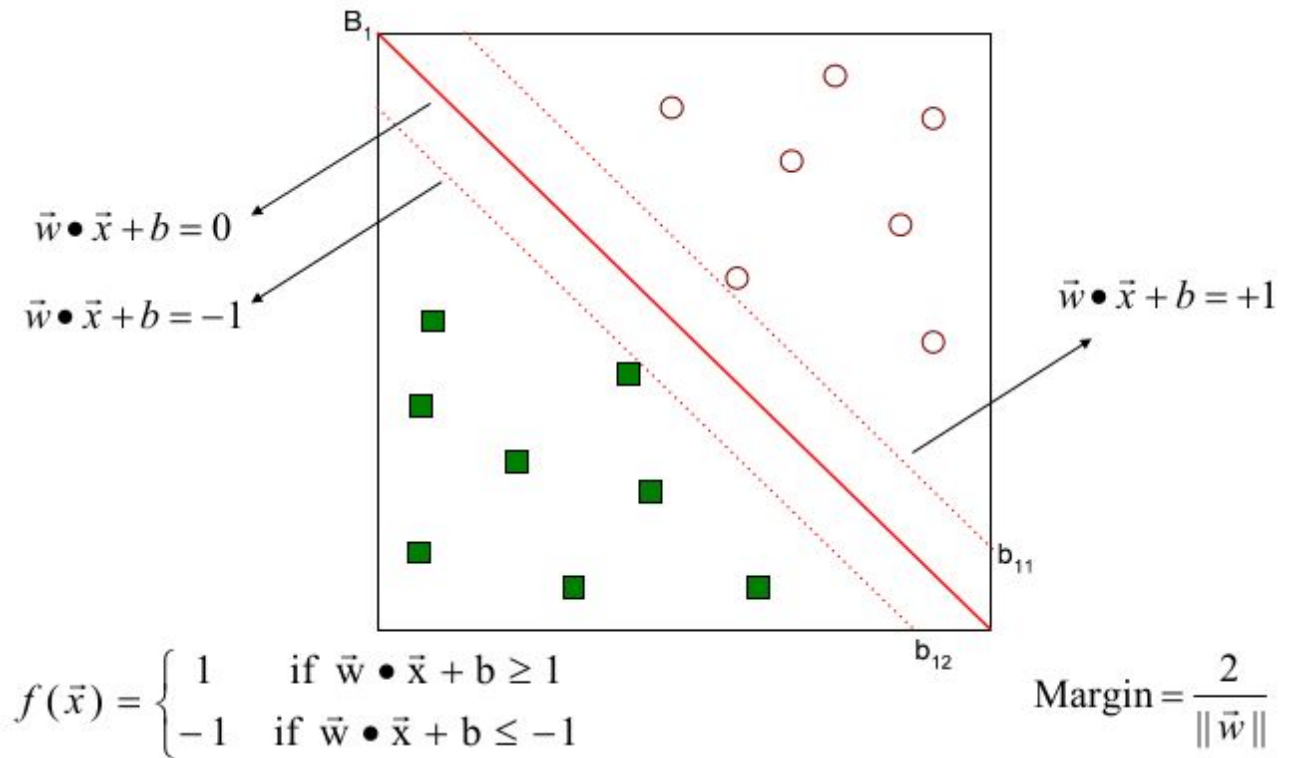
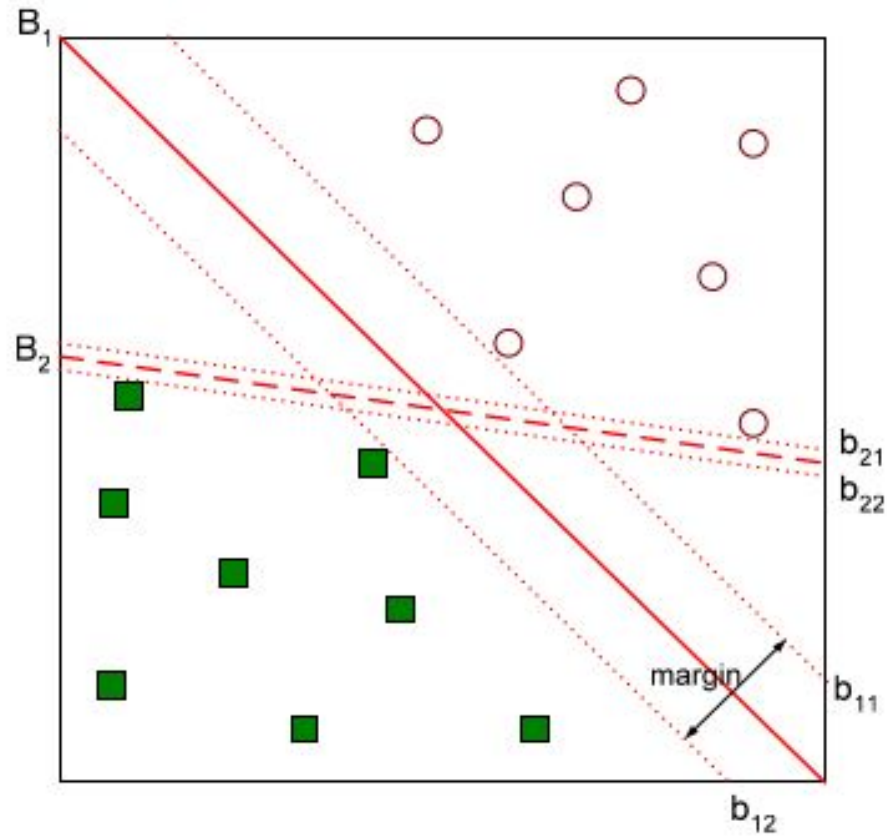
Máquina de Vetores Suporte - SVM



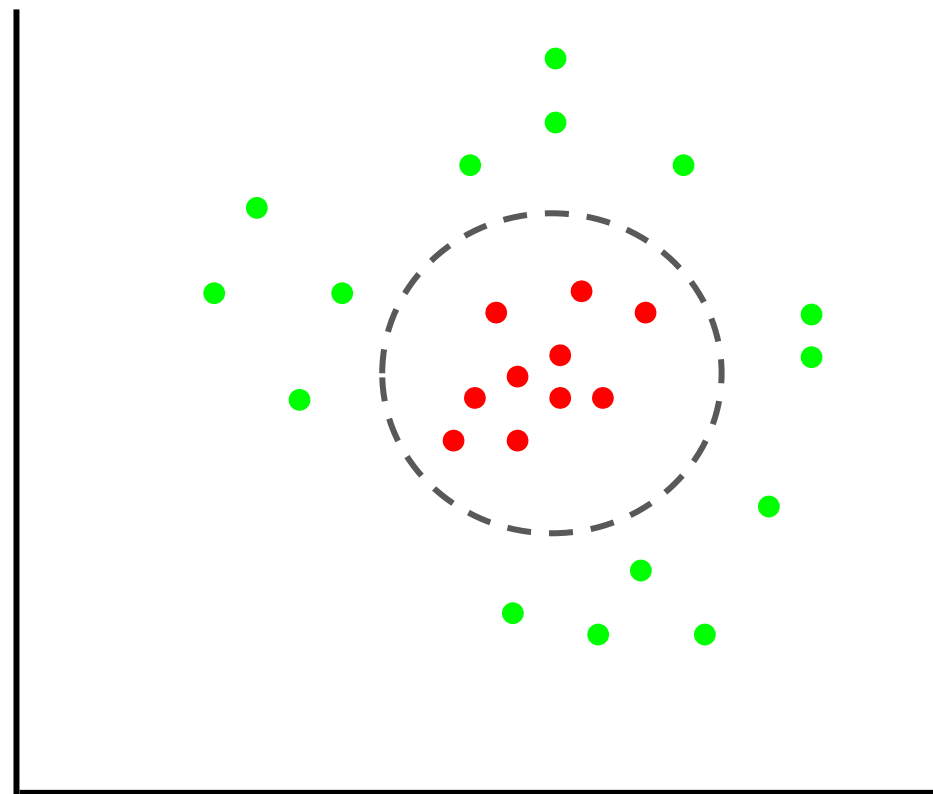
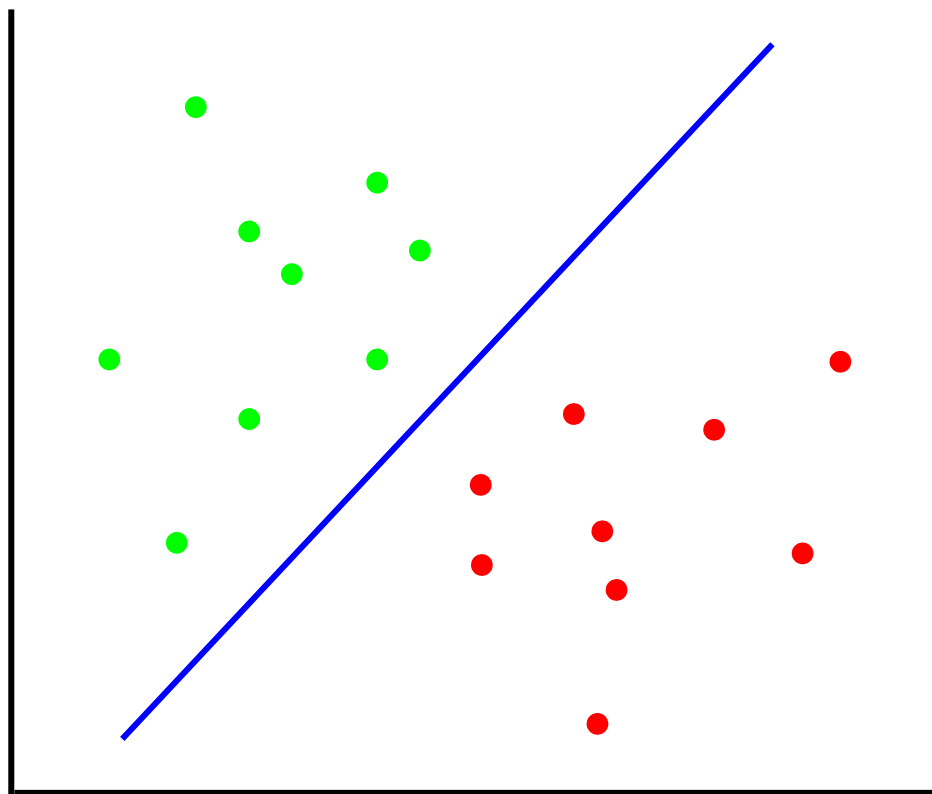
Máquina de Vetores Suporte - SVM



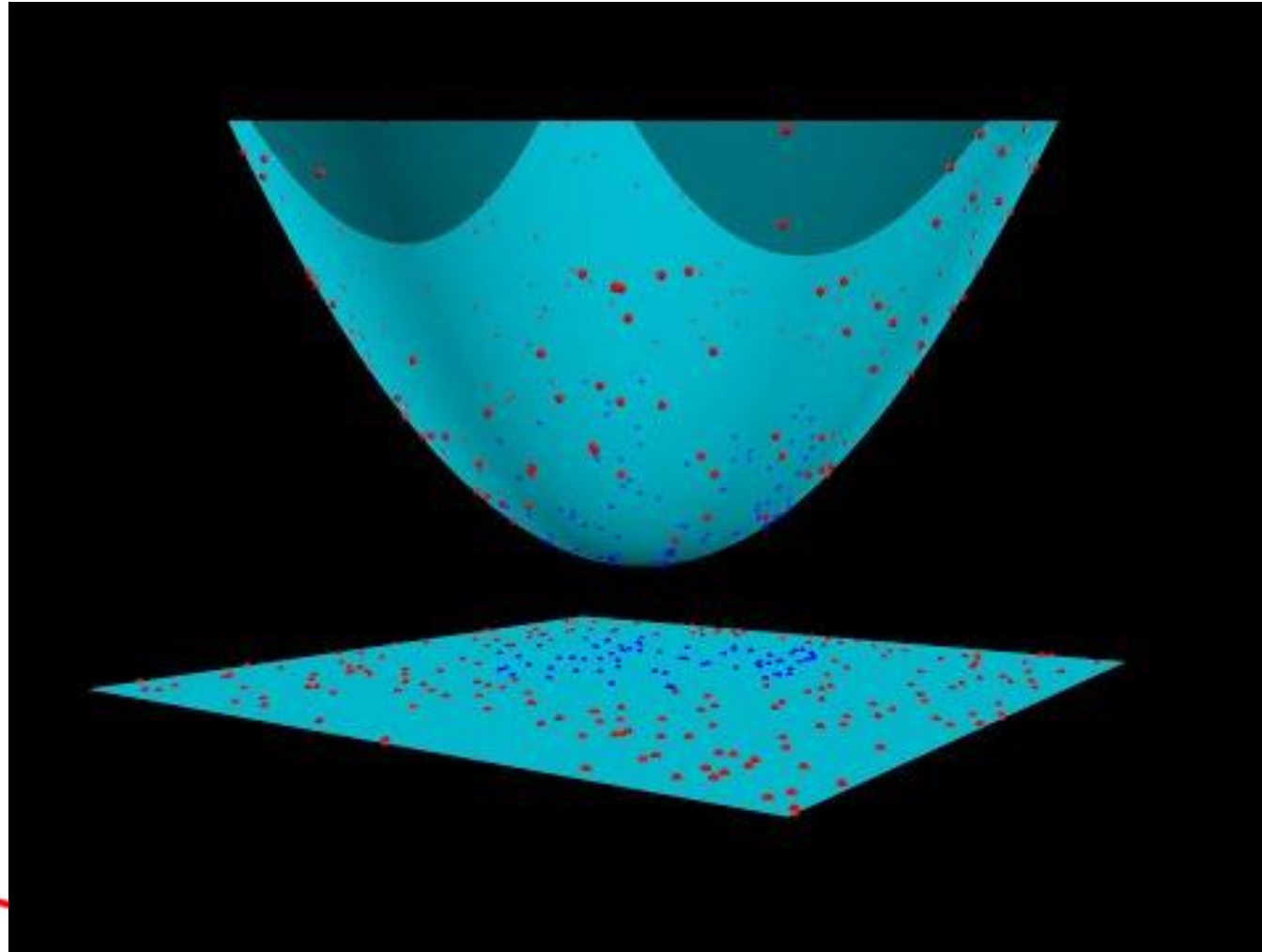
Máquina de Vetores Suporte - SVM



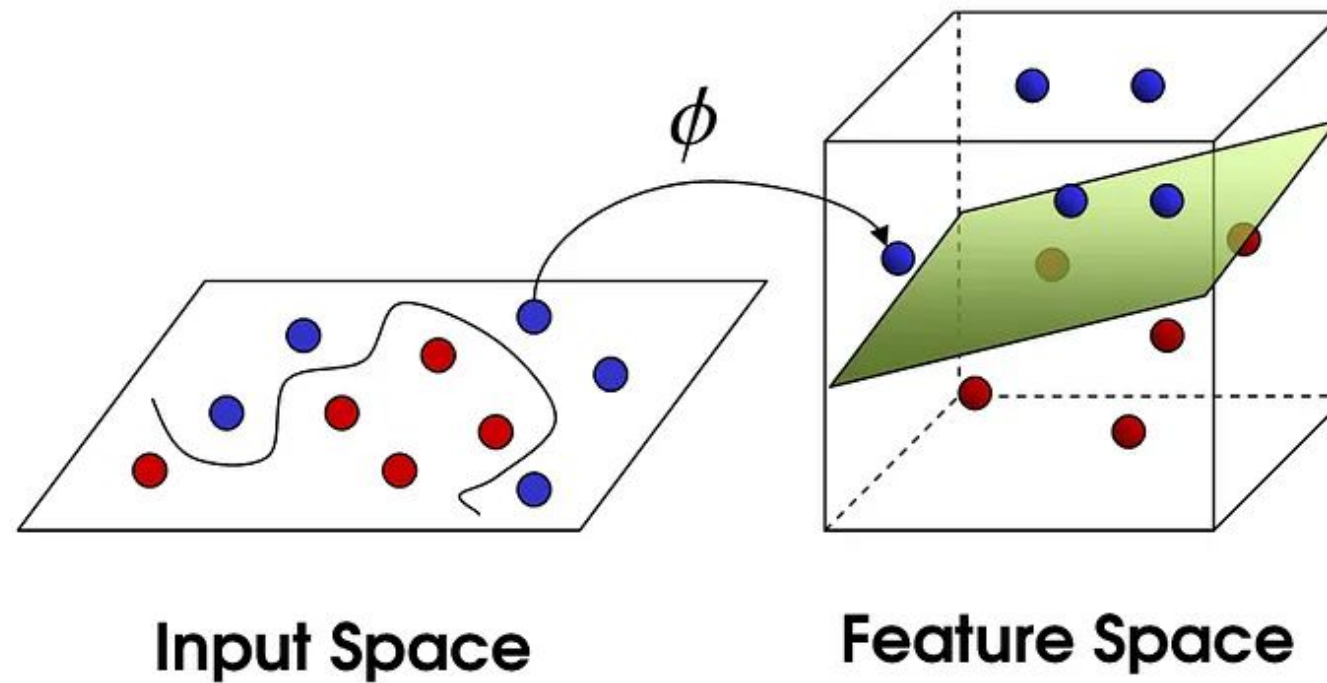
SVM - “Truque” do Kernel



SVM - “Truque” do Kernel



SVM - “Truque” do Kernel



Máquina de Vetores Suporte - SVM

$$\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ x_{31} & x_{32} & \cdots & x_{3n} \\ x_{41} & x_{42} & \cdots & x_{4n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ \vdots \\ y_m \end{bmatrix}$$

$\mathbf{X} \quad \theta \quad = \quad \mathbf{y}$

Máquina de Vetores Suporte - SVM

$$\begin{aligned}\mathbf{X}\theta &= \mathbf{y} \\ \mathbf{X}^\top \mathbf{X}\theta &= \mathbf{X}^\top \mathbf{y} \\ \theta &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}\end{aligned}$$

Máquina de Vetores Suporte - SVM

$$h(\mathbf{x}) = \sum_{i=0}^n \theta_i x_i = \theta^\top \mathbf{x}.$$

$$h(\mathbf{x}) = \sum_{i=0}^n \theta_i \phi(x_i) = \theta^\top \phi.$$



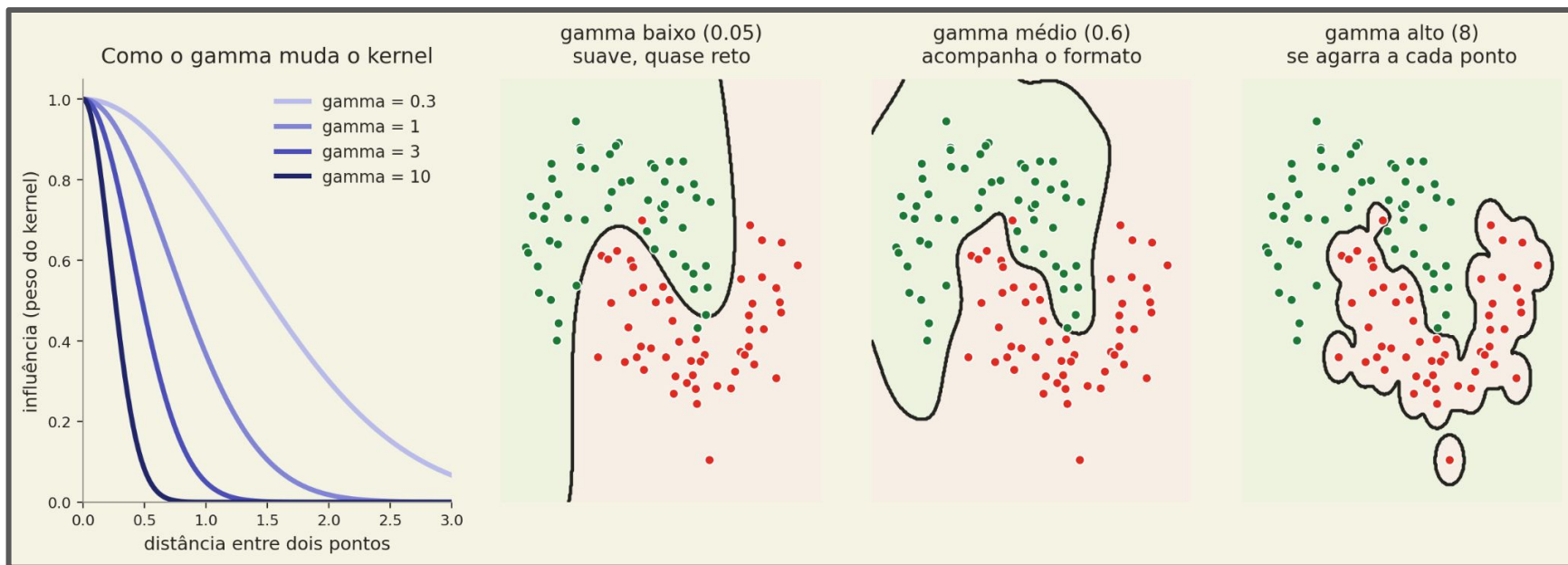
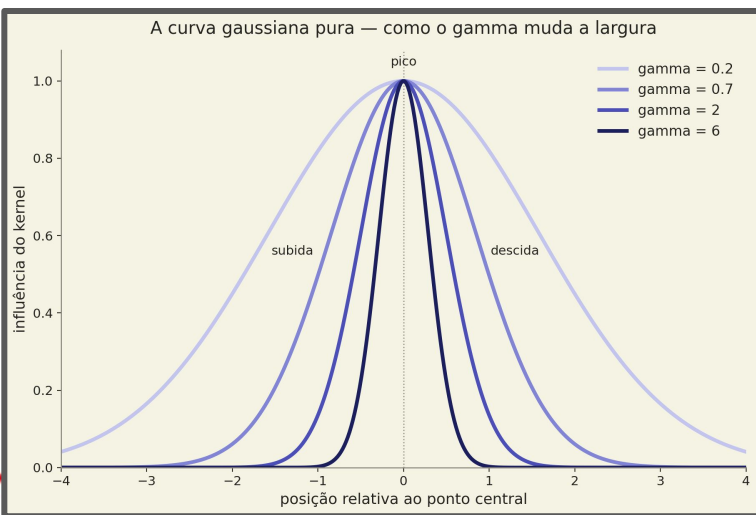
Máquina de Vetores Suporte - SVM

$$k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}, \gamma > 0$$

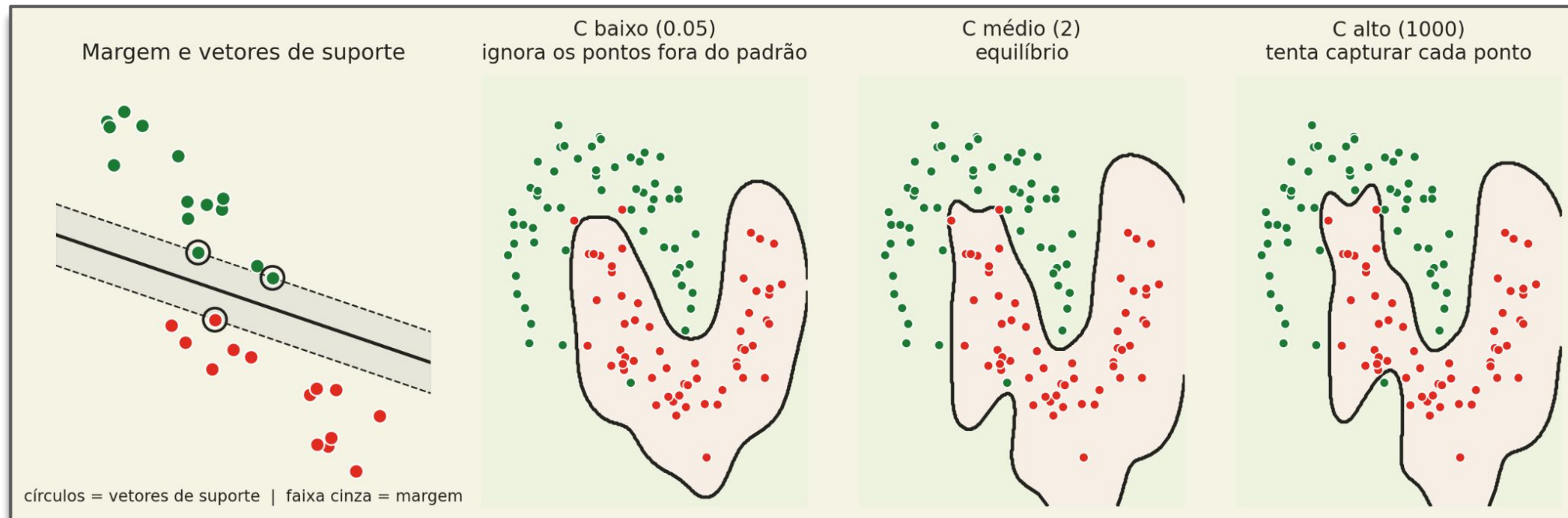


Máquina de Vetores Suporte - SVM

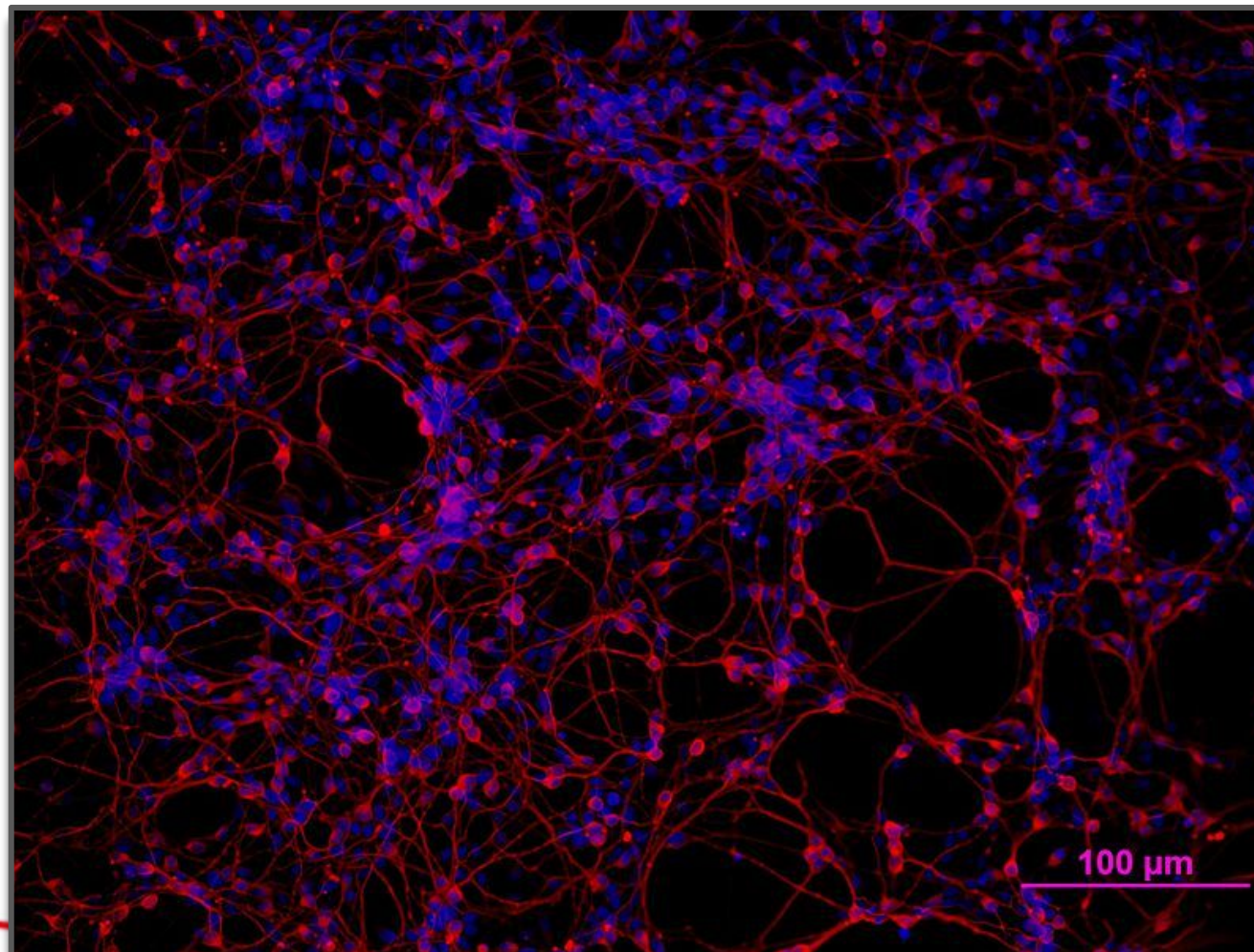
$$k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}, \gamma > 0$$



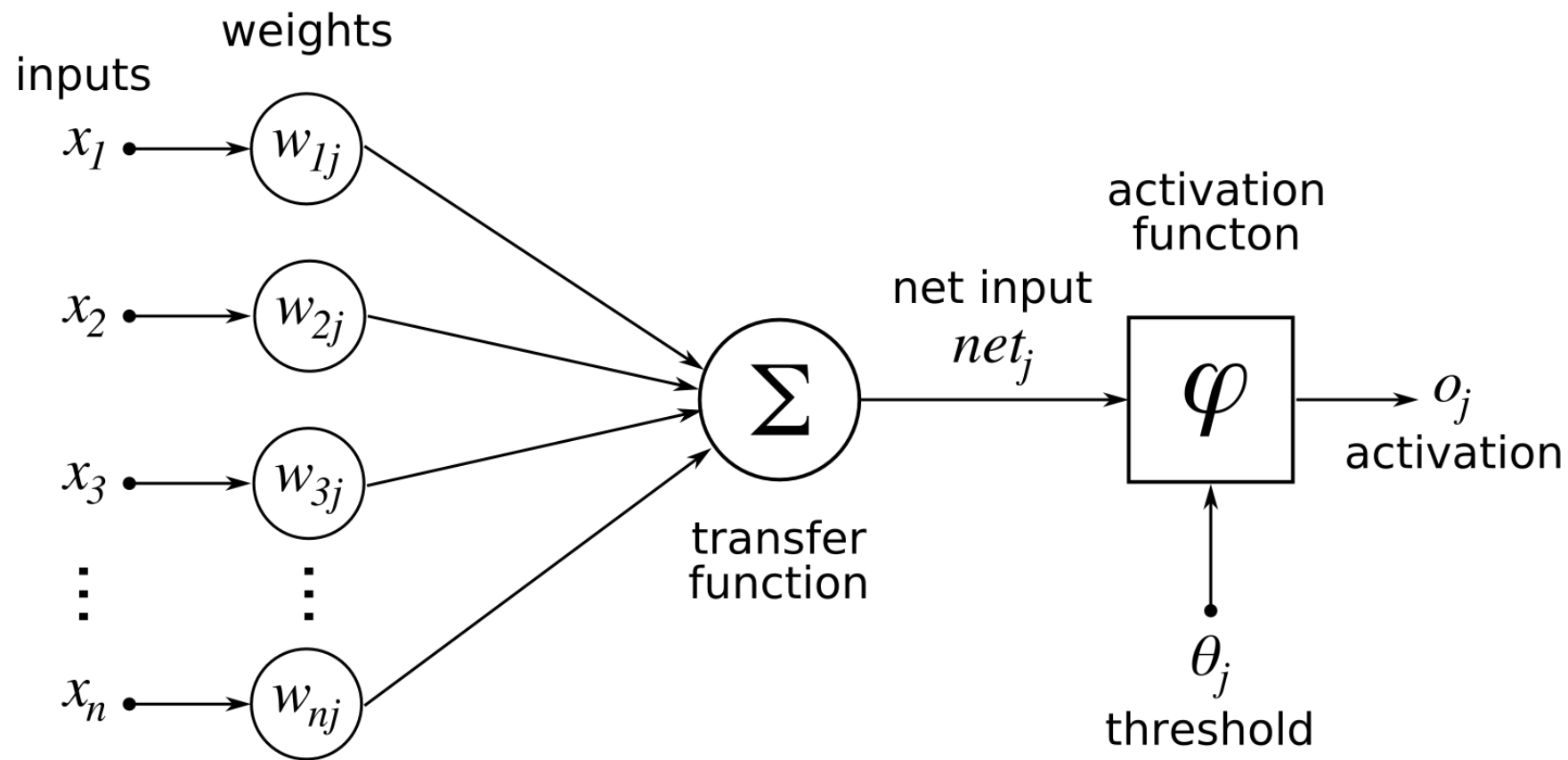
Máquina de Vetores Suporte - SVM



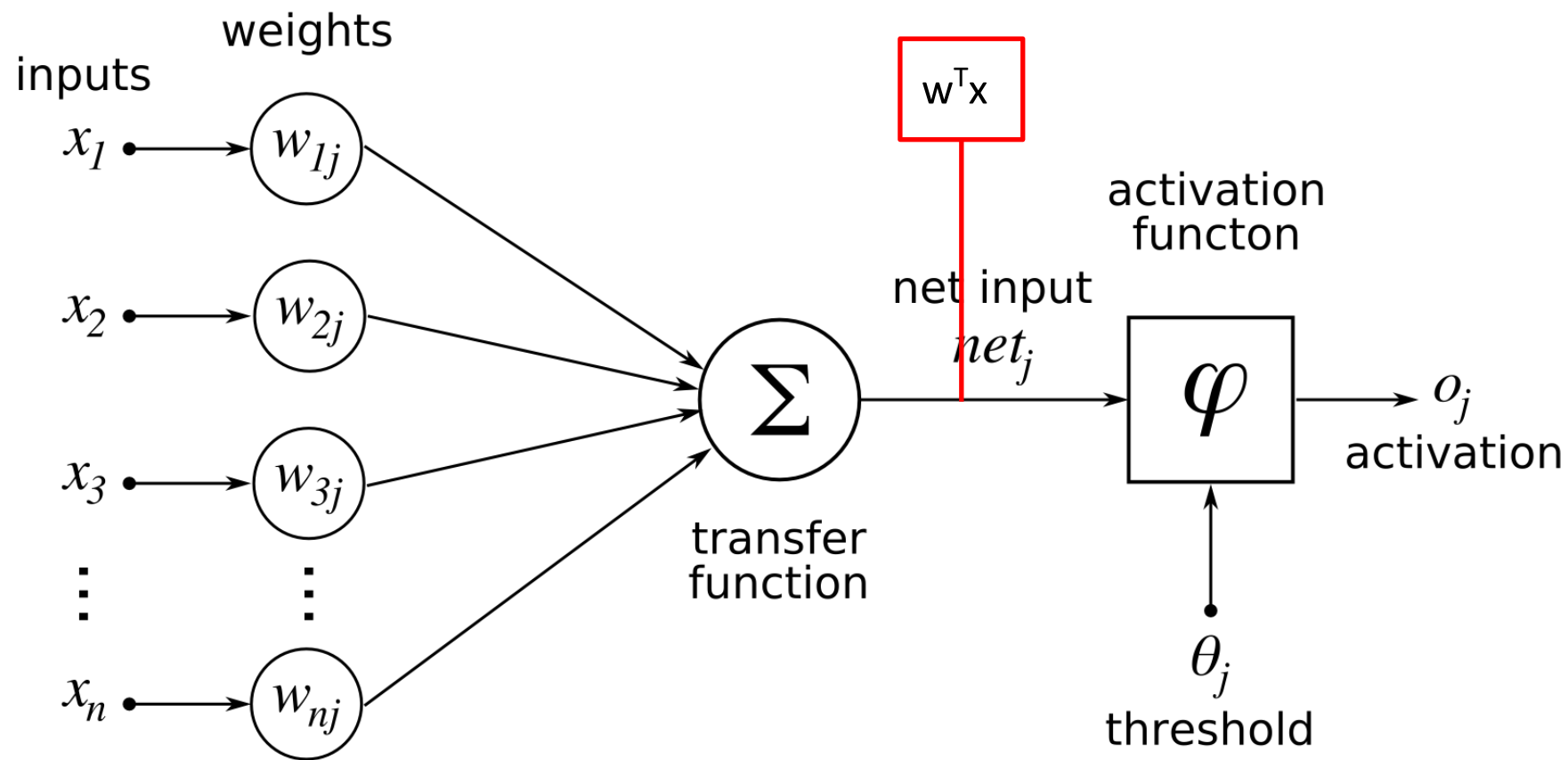
Redes Neurais Artificiais - RNA



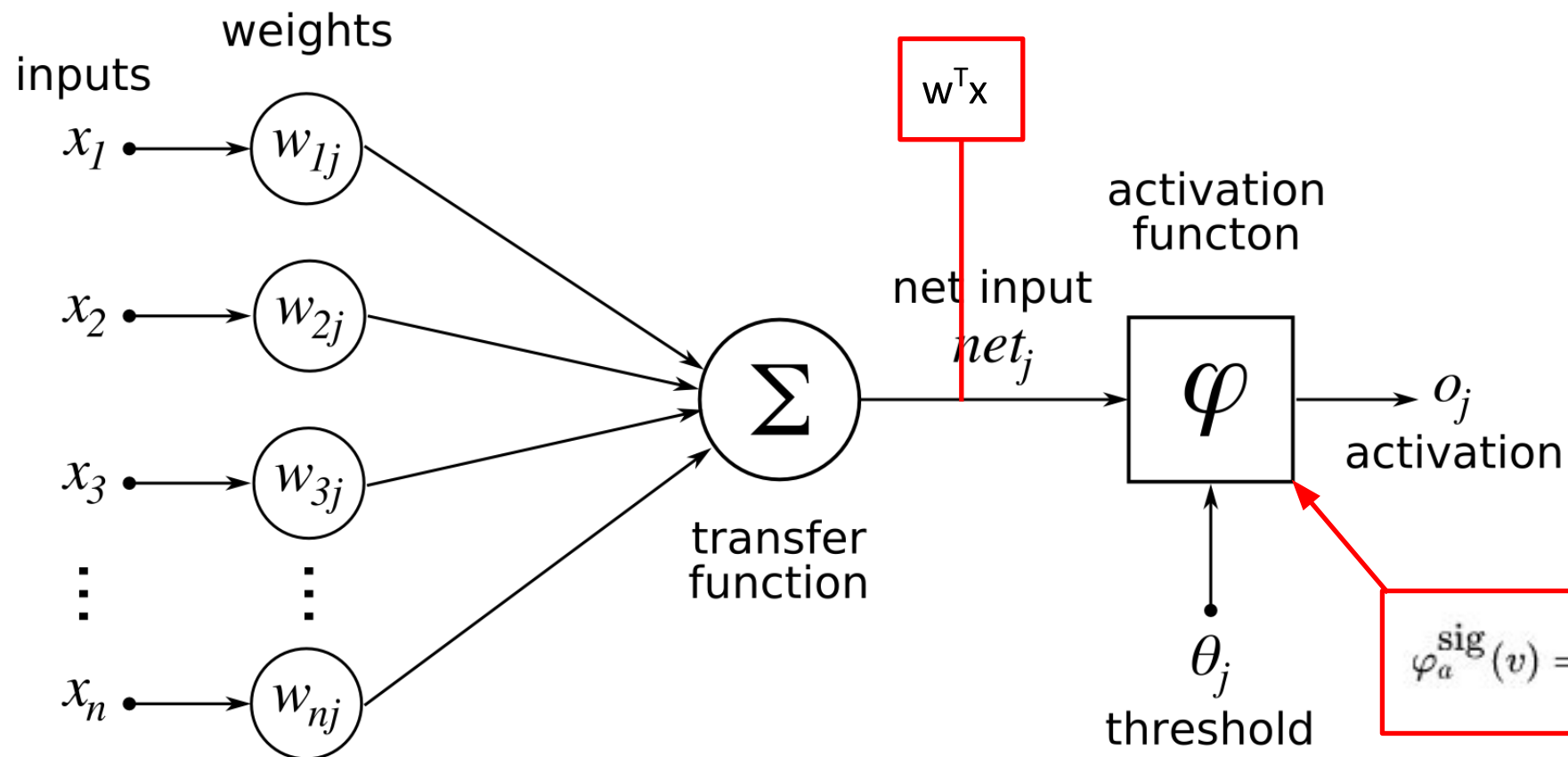
Redes Neuronais Artificiais - RNA



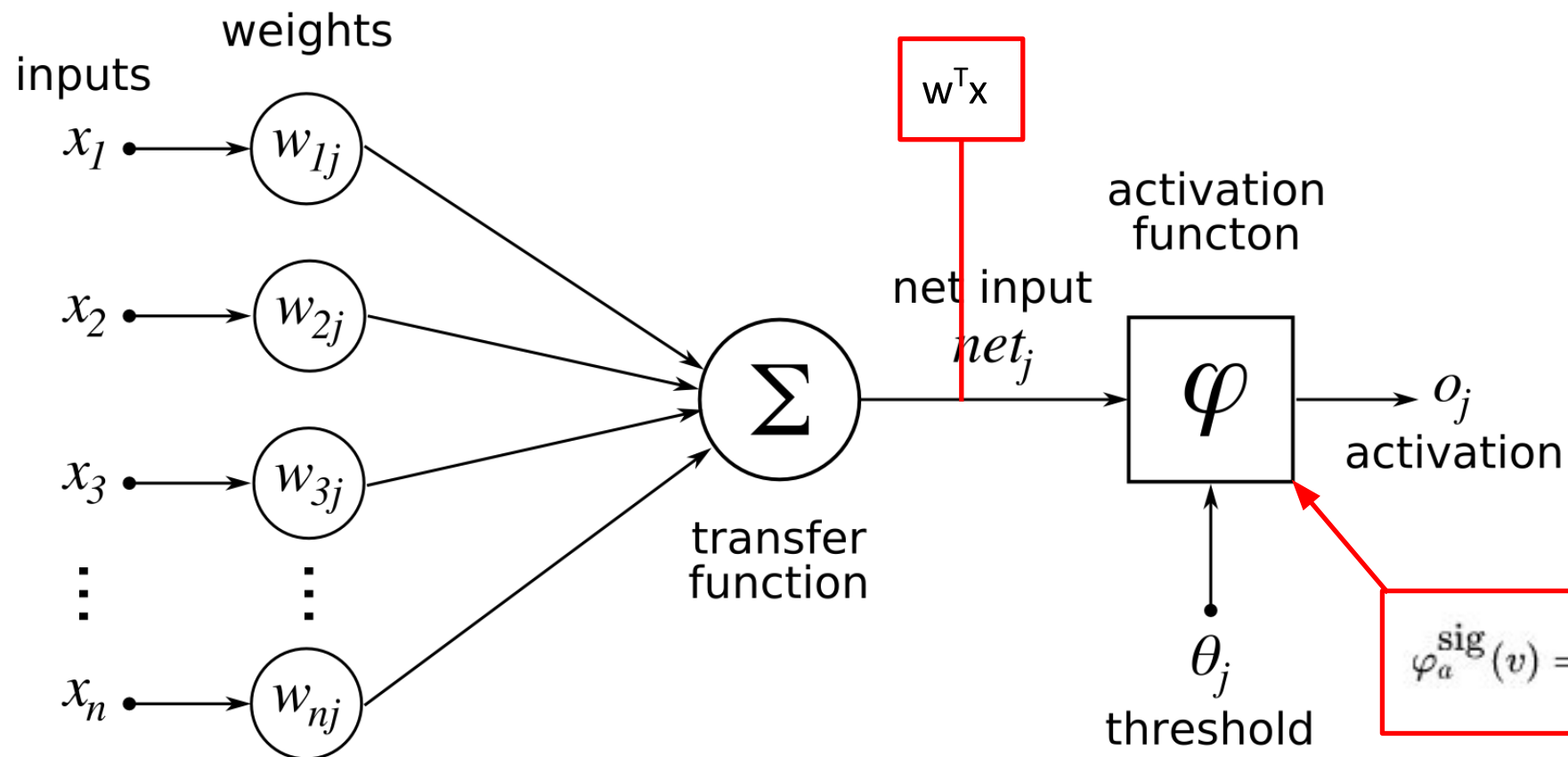
Redes Neuronais Artificiais - RNA



Redes Neuronais Artificiais - RNA

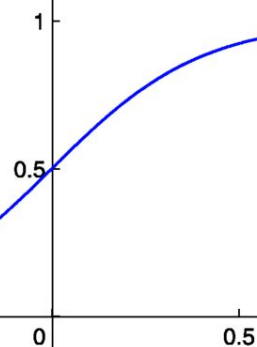


Redes Neuronais Artificiais - RNA

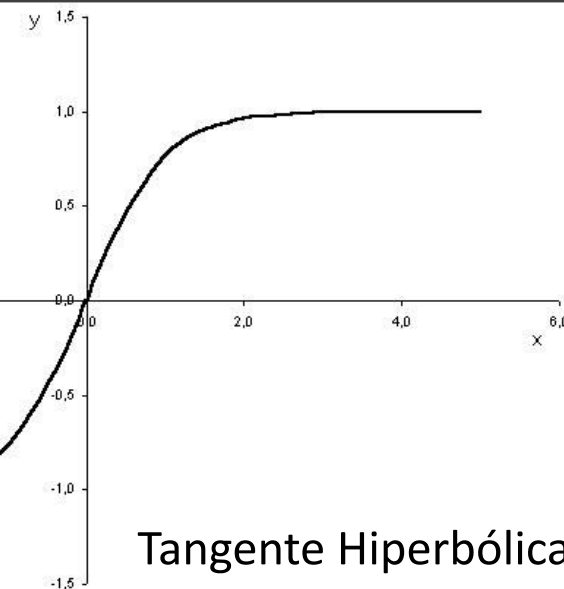
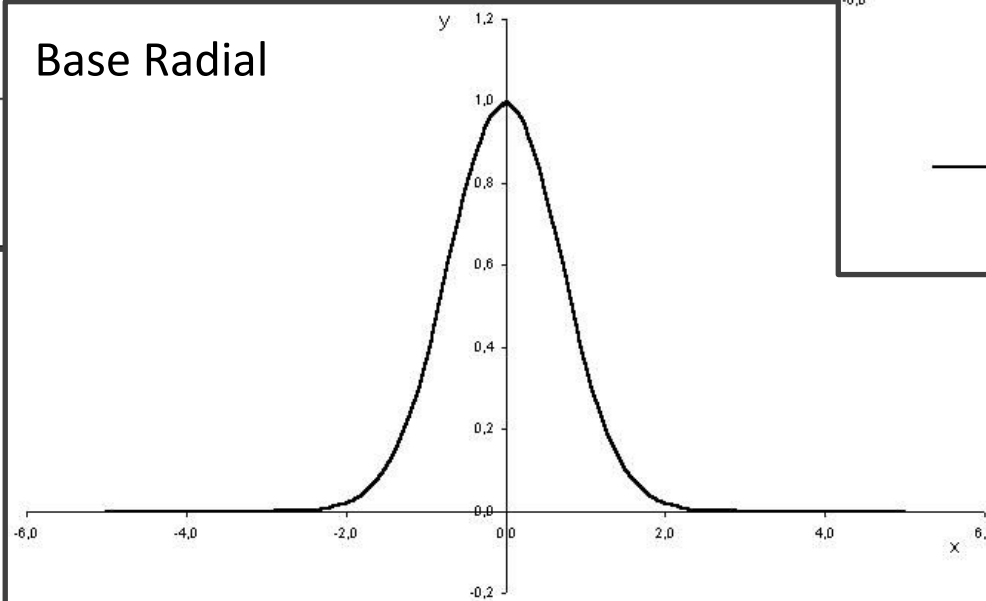


Redes Neuronais Artificiais - RNA

Logística

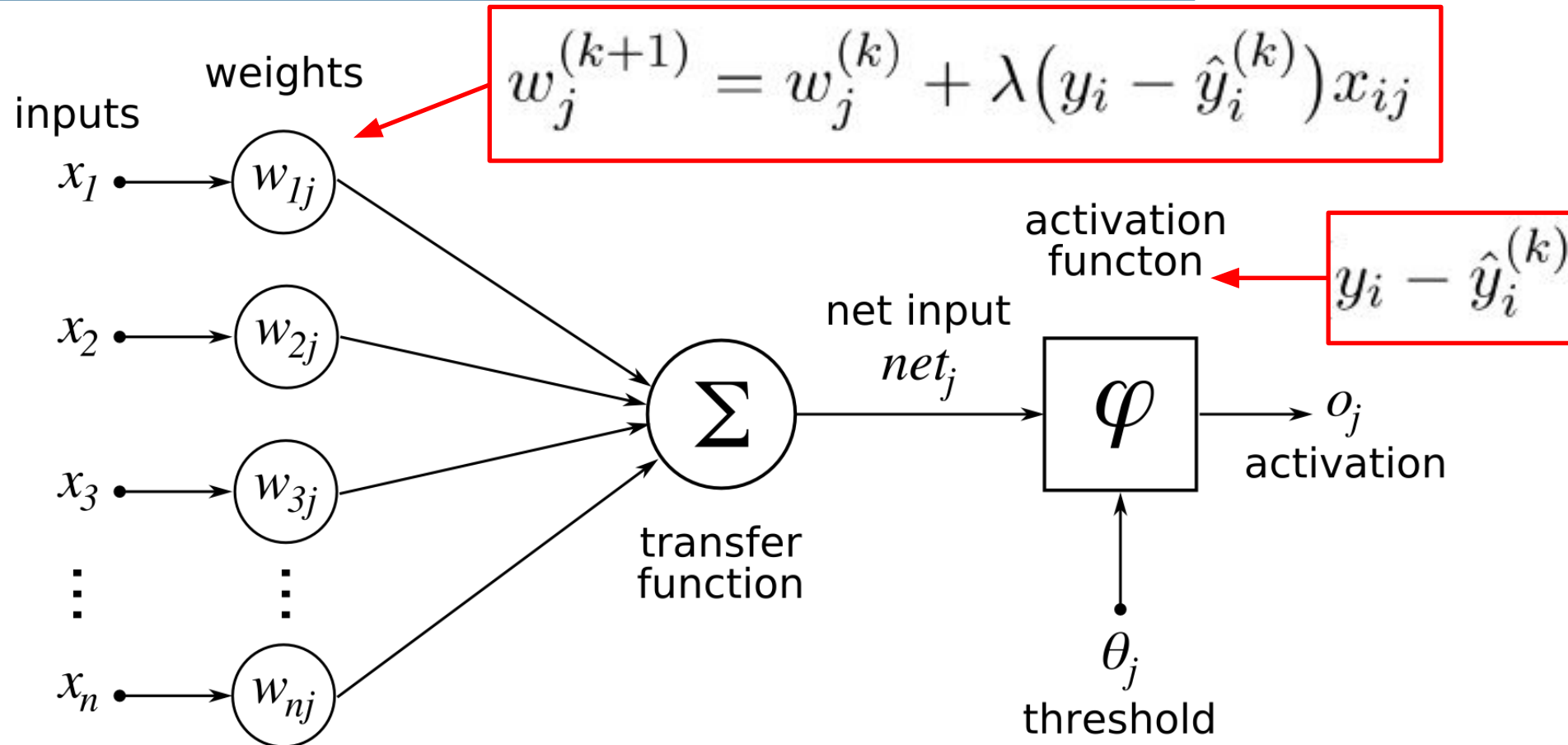


Base Radial

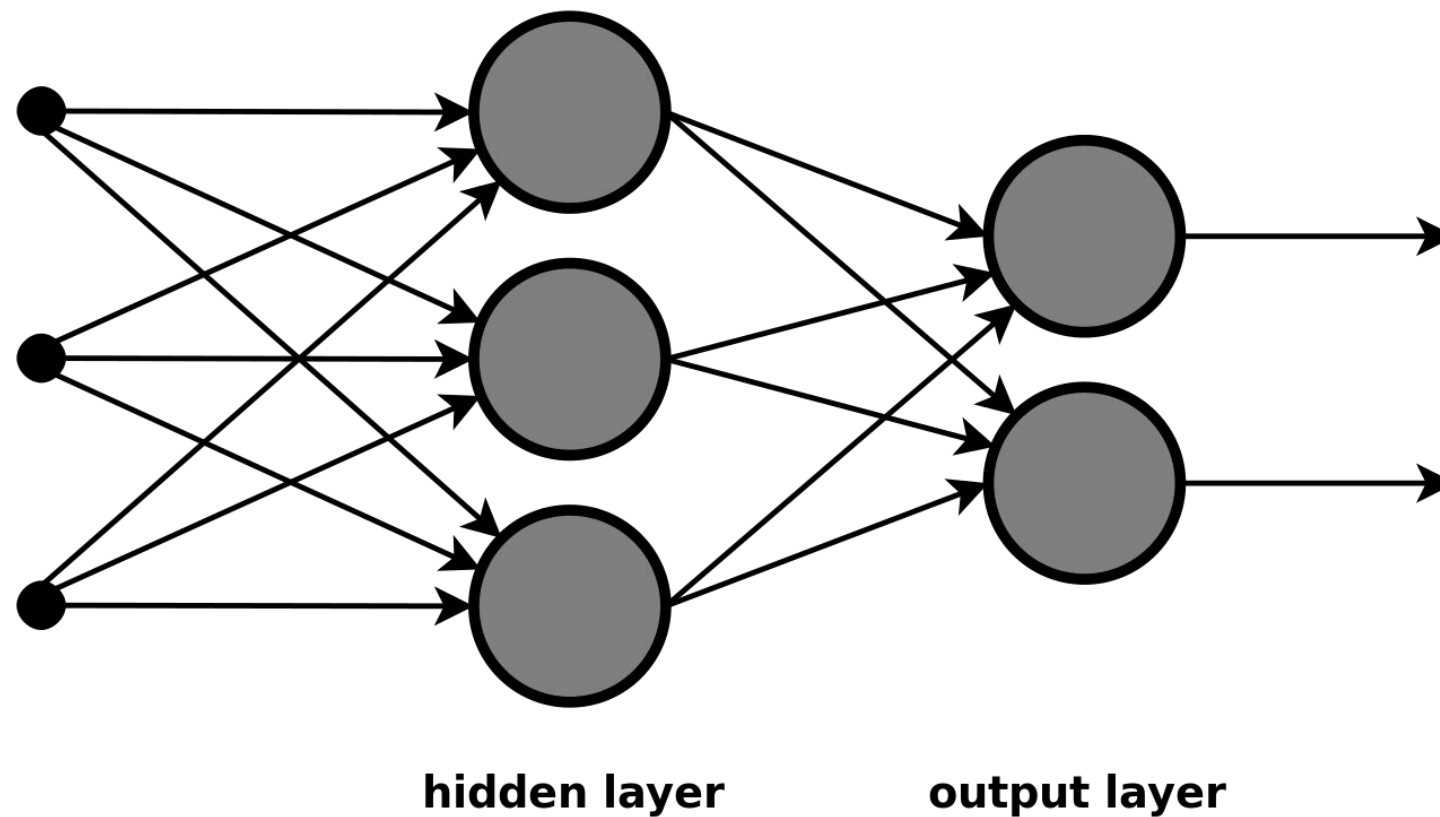


Tangente Hiperbólica

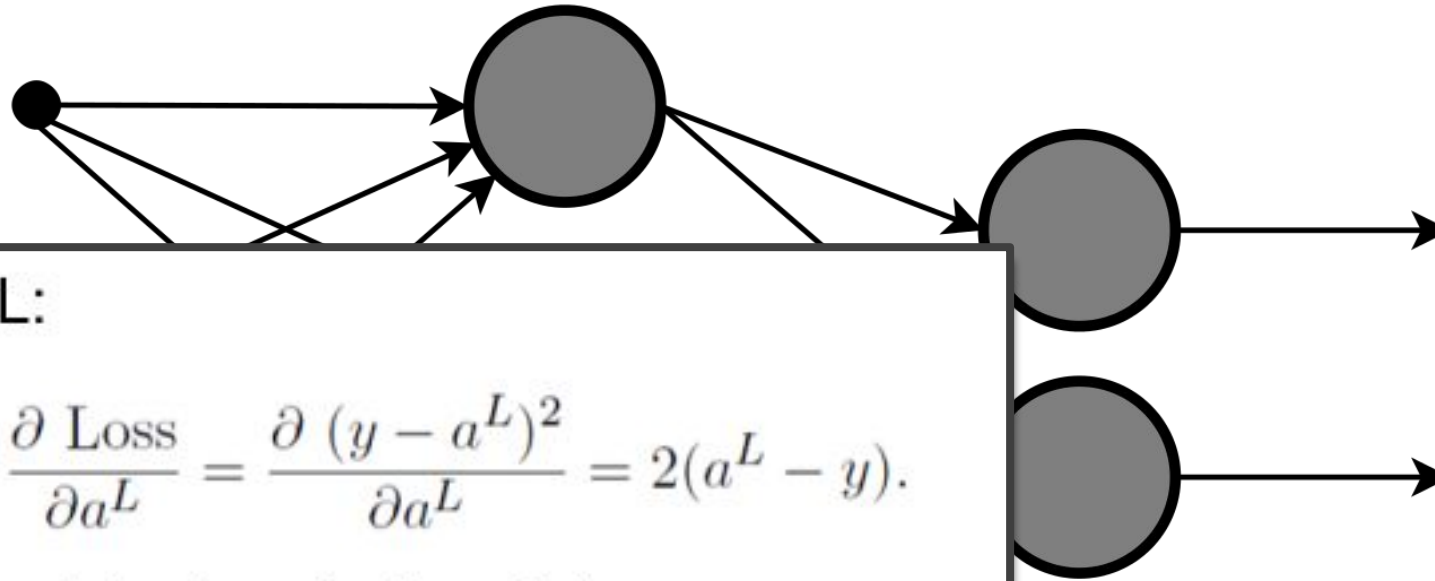
Redes Neuronais Artificiais - RNA



Redes Neurais Artificiais - RNA



Redes Neurais Artificiais - RNA



- At output layer L:

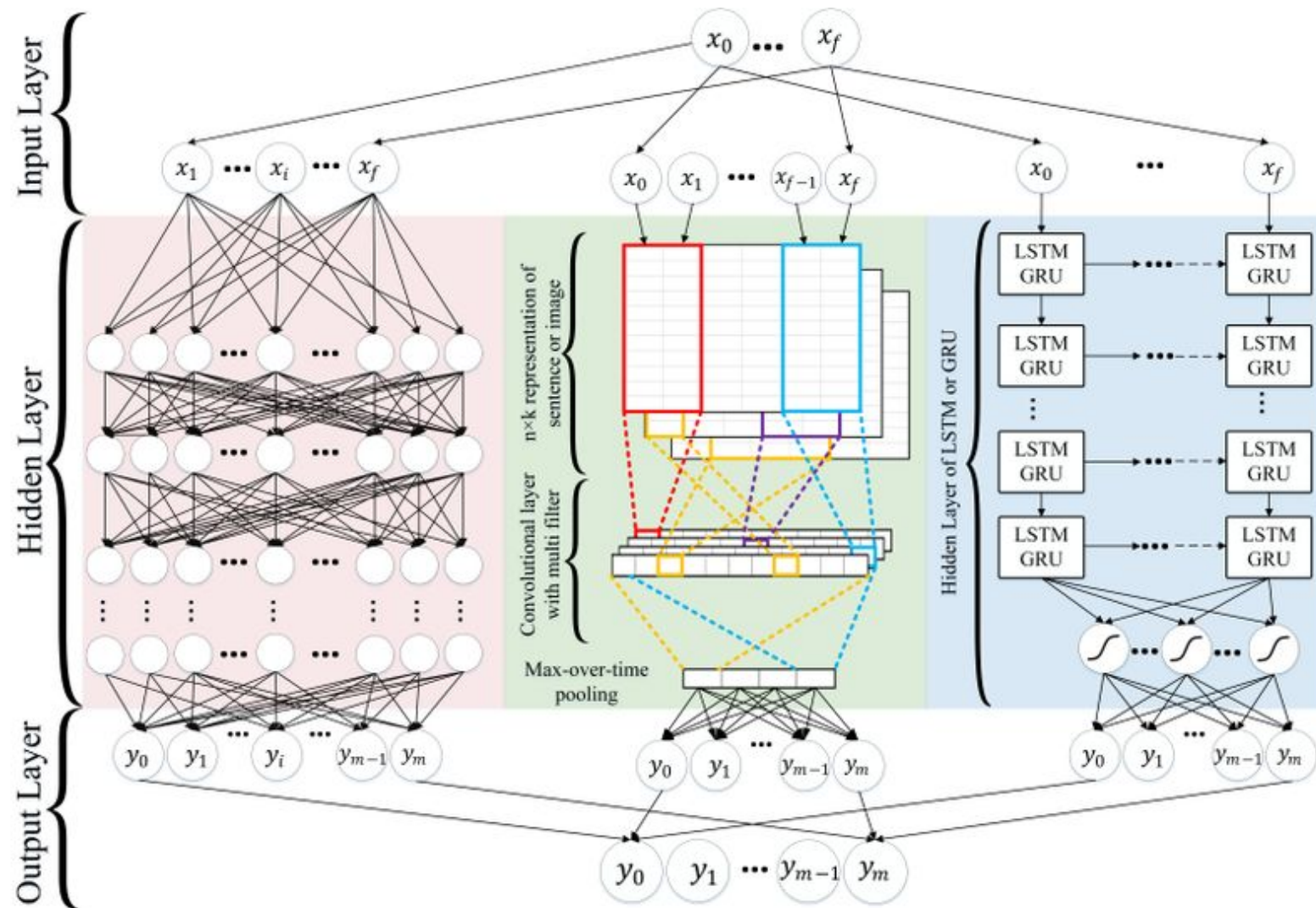
$$\delta^L = \frac{\partial \text{Loss}}{\partial a^L} = \frac{\partial (y - a^L)^2}{\partial a^L} = 2(a^L - y).$$

- At a hidden layer l (using chain rule):

$$\delta_j^l = \sum_i (\delta_i^{l+1} \times a_i^{l+1} (1 - a_i^{l+1}) \times w_{ij}^{l+1}).$$

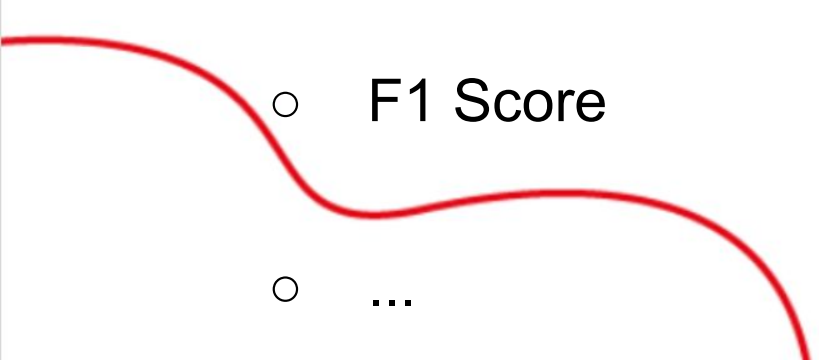
out layer

"Deep Learning"





Como selecionar modelos?

- Medidas para avaliação de modelos:
 - Acurácia
 - Matriz de confusão
 - Precisão (precision)
 - Revocação (Recall)
 - F1 Score
 - ...
- 



Como selecionar modelos?

Classes	Verdadeira Positivo (P)	Verdadeira Negativo (N)
Prevista Positivo (P)	TP	FP
Prevista Negativo (N)	FN	TN

True

False

Como selecionar modelos?

- Medidas para avaliação de modelos:

- Acurácia

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- Matriz de confusão

- Precisão (precision)

$$\frac{TP}{TP + FP}$$

- Revocação (Recall)

$$\frac{TP}{TP + FN}$$

- F1 Score

$$\frac{2 \times (\text{Precisão} \times \text{Revocação})}{\text{Precisão} + \text{Revocação}}$$

- ...

Como selecionar modelos?

- Curva ROC e Área sob a curva ROC
 - ROC: Receiver Operating Characteristic
 - Gráfico que mostra o desempenho de um modelo de classificação para todos os limiares de discriminação (*thresholds*). Possui dois parâmetros:
 - Taxa de Verdadeiros Positivos (True Positive Rate - TPR)
 - Taxa de Falsos Positivos (False Positive Rate - FPR)

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Como

- Curva ROC

- ROC

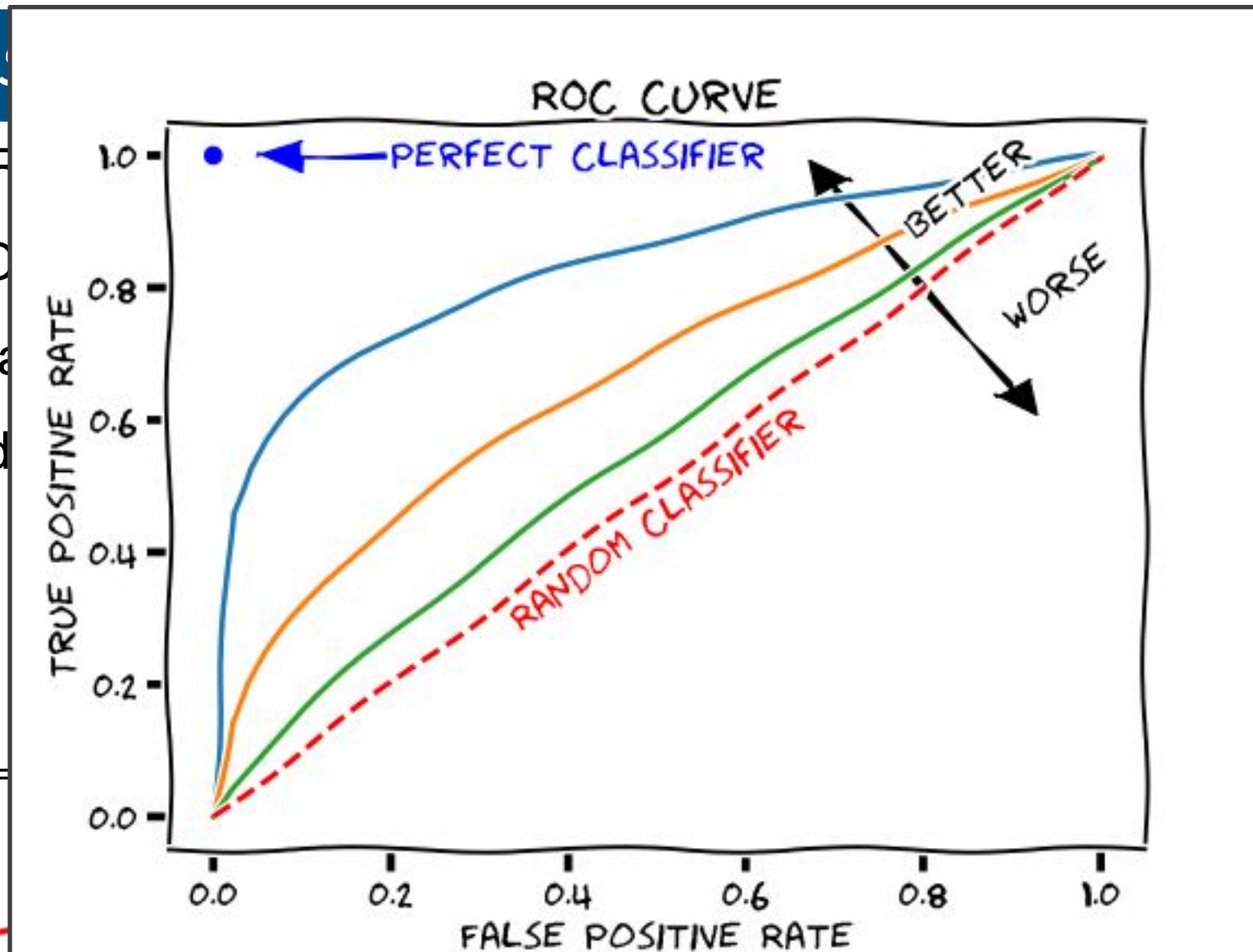
- Gra

tod

■

■

TPR =

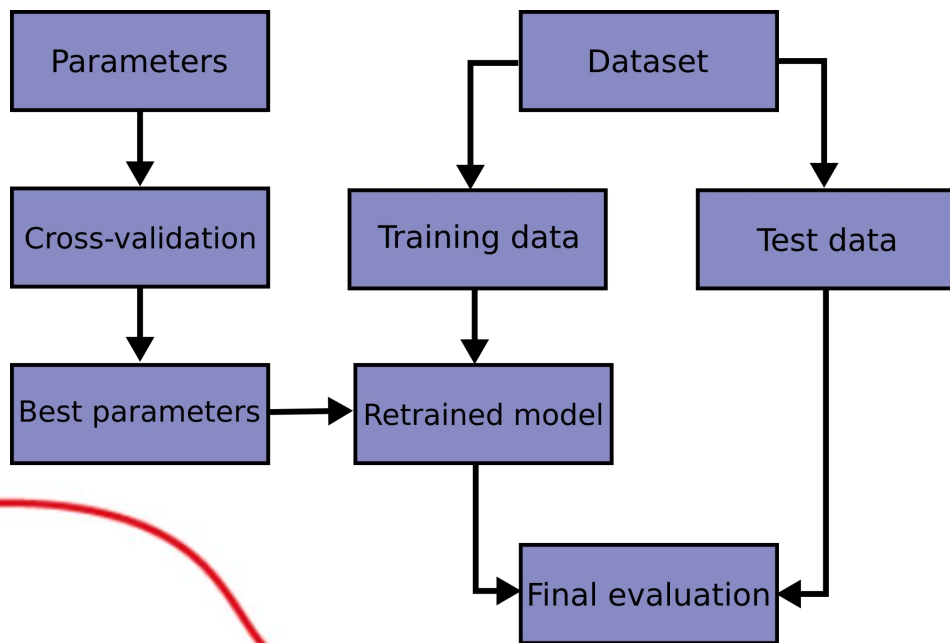


sificação para
os parâmetros:
(TPR)

TN

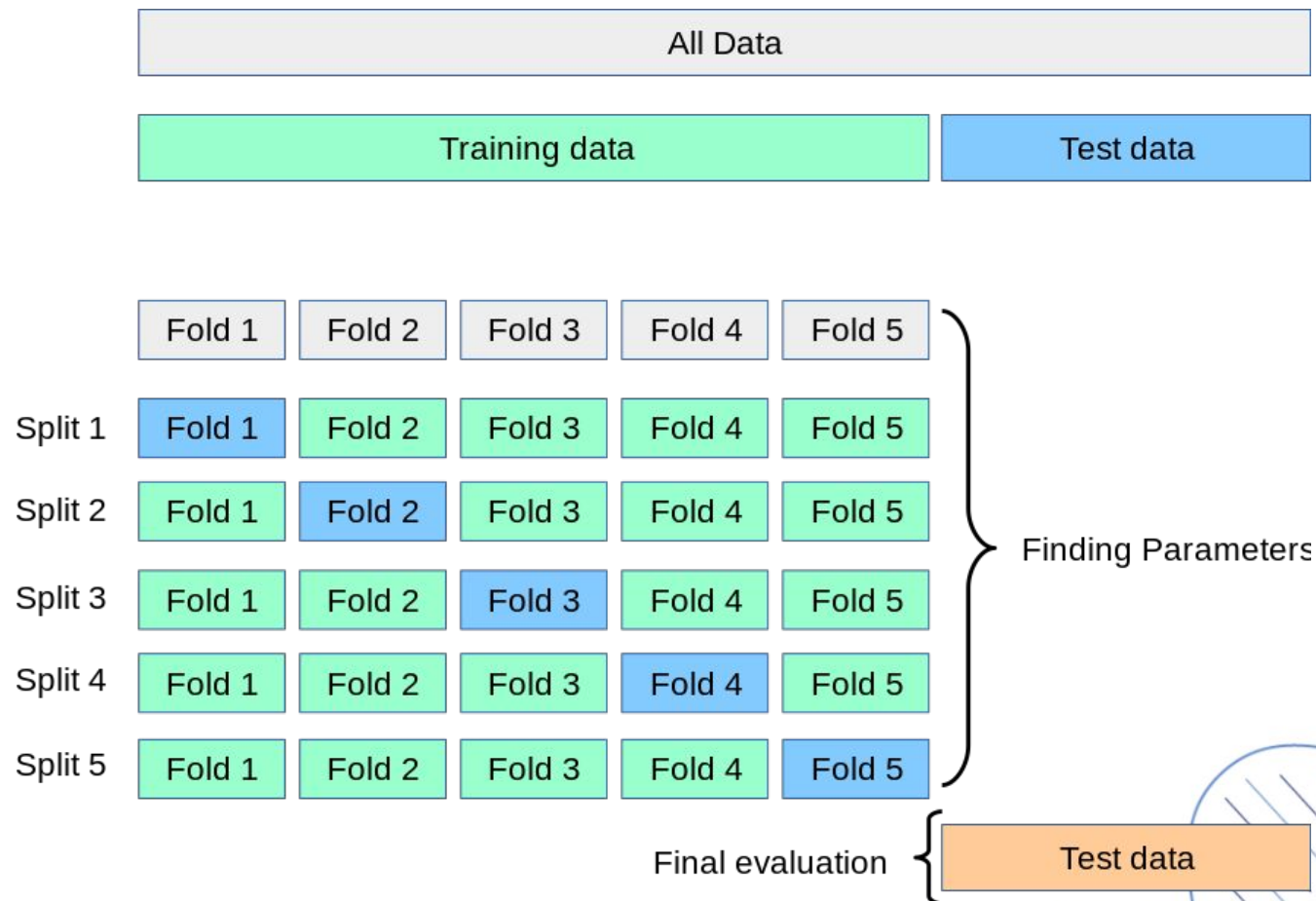
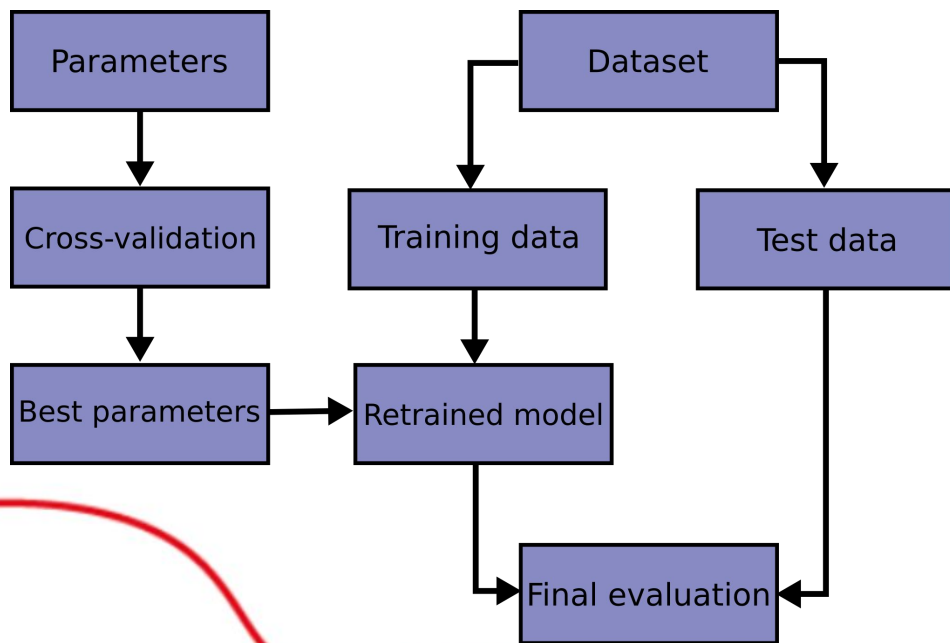
Validação Cruzada

- Buscar por melhores parâmetros (seleção de modelos) sem afetar a avaliação final do modelo



Validação Cruzada

- Buscar por melhores parâmetros (seleção de modelos) sem afetar a avaliação final do modelo



Obrigado

Eduardo Krempser
Assessoria de Inteligência Competitiva
Bio-Manguinhos/Fiocruz

Pedro Henrique Monteiro Torres
Instituto de Biofísica Carlos Chagas Filho - IBCCF
Universidade Federal do Rio de Janeiro - UFRJ

